

摘 要

随着企业信息化建设的不断深入，传统依赖人工统计、分散表格和线下沟通的管理方式已难以满足企业级场景下项目协同、质量跟踪和运营分析的需求。针对某汽车电子企业现有项目管理工作中数据分散、缺少统一视图的问题，问题跟踪效率不高等情况，本文围绕该企业数字化运营平台的升级需求，旨在通过扩展公司管理与项目管理相关功能，提升企业项目过程管控和运营数据分析能力。

结合当前企业级应用的发展趋势，本文基于 B/S 架构完成系统实现。后端采用 Spring Boot 框架进行开发，前端采用 SPA 单页面应用技术实现。通过分析企业软件项目管理的日常工作流程，本文围绕部门投入 BU 工时分布、项目基本信息管理、缺陷管理和重大问题管理四个模块进行功能扩展，并结合权限控制、飞书消息通知等，完成了业务数据的统一管理、项目质量问题的跟踪以及运营数据的统计分析。同时对性能做了一些优化。

目前该系统已在公司内部部署使用，能够支持企业对项目基础信息、质量问题和缺陷数据进行统一管理，并接入飞书平台。系统减少了数据分散带来的管理成本，提升了企业项目管理的信息化水平，增强了项目管理过程中的质量规范和运营分析能力。

关键词：汽车电子软件，数字化运营平台，项目管理，质量跟踪，B/S 架构

ABSTRACT

Many enterprise project management workflows still rely on manual statistics, separate spreadsheets, and offline coordination, which makes it difficult to maintain a consistent view of project progress, quality issues, and operational data. This thesis addresses the upgrade of a digital operation platform used by an automotive electronics enterprise. The existing workflow suffers from fragmented project data, limited cross-module visibility, and inefficient issue tracking. To address these problems, this thesis extends the platform's company management and project management functions, so as to support project process control and operational data analysis in a more integrated manner.

The system is implemented with a B/S architecture. Spring Boot is used to build the back-end services, while the front end is developed as a single-page application. Based on an analysis of the enterprise's daily software project management workflow, this work extends four functional modules: department-to-BU work-hour distribution, project basic information management, defect management, and major issue management. Permission control and Feishu-based message notification are also incorporated. These functions support unified business data management, project quality issue tracking, operational statistics, and performance optimization for selected query scenarios.

The upgraded system has been deployed for internal use. It provides unified management of project basic information, quality issues, and defect data, and connects relevant workflow notifications to Feishu. In practical use, the system reduces the management cost caused by scattered data sources, improves the informatization level of enterprise project management, and provides better support for quality control and operational analysis.

KEY WORDS: Automotive Electronic Software, Digital Operation Platform, Project Management, Quality Tracking, B/S Architecture

目 录

摘 要	1
ABSTRACT	2
目 录	3
第一章 绪论	1
1.1 研究背景和意义	1
1.2 国内外研究现状	1
1.3 论文的主要内容	3
1.4 论文的组织结构	3
第二章 相关开发技术介绍	5
2.1 SpringBoot 框架	5
2.1.1 SpringBoot 框架核心理念	5
2.1.2 SpringBoot 框架关键特性	5
2.2 Vue 框架	6
2.3 数据库技术	7
2.3.1 关系型数据库 MySQL	7
2.3.2 非关系型数据库 Redis	8
2.4 MyBatis-Plus	9
2.5 本章小结	9
第三章 需求分析	10
3.1 需求概述	10
3.2 功能需求	10
3.2.1 部门投入 BU 工时分布需求分析	11
3.2.2 项目基本信息管理需求分析	11
3.2.3 缺陷管理需求分析	12
3.2.4 重大问题管理需求分析	13
3.3 非功能需求	14
3.3.1 性能要求	14
3.3.2 接口可靠性	14
3.3.3 易用性	15
3.4 本章小结	15
第四章 系统总体设计	16
4.1 系统整体架构设计	16
4.2 系统功能模块划分	18
4.3 本章小结	19
第五章 核心业务模块的详细设计与实现	20
5.1 模块设计	20

5.1.1 部门投入 BU 工时分布.....	20
5.1.2 项目基本信息管理.....	23
5.1.3 缺陷管理	25
5.1.4 重大问题管理.....	27
5.2 数据库设计	28
5.2.1 概念逻辑设计...逻辑结构设计.....	29
5.2.2 数据表设计	30
5.3 本章小结	33
第六章 系统测试	34
6.1 测试环境	34
6.2 功能测试	34
6.2.1 部门投入 BU 工时分布功能测试.....	34
6.2.2 项目基本信息管理测试	36
6.2.3 缺陷管理测试.....	37
6.2.4 重大问题管理测试	38
6.2.5 各功能的边界测试	40
6.3 非功能测试	40
6.3.1 安全性测试	40
6.3.2 性能测试	40
6.4 本章小结	42
第七章 总结与展望	43
7.1 工作总结	43
7.2 工作展望	43
参考文献	45

第一章 绪论

1.1 研究背景和意义

缩写改为英文全称

随着全球汽车工业加速迈向数字化与智能化，“软件定义汽车”（SDV）这个理念已逐步成为下一阶段汽车产业的演进方向^[1]。在这一趋势下，车辆的创新焦点和核心价值逐步由传统的机械底盘与动力系统，转移至底层的中央计算平台与上层的高阶软件架构。软件不仅能够影响智能座舱的交互体验与智能驾驶的安全边界，更赋予了车辆在全生命周期内通过无线（OTA）技术持续进化、按需解锁新功能的能力^[2]。然而，这种软件规模与复杂度的爆炸式增长，也给汽车电子研发带来了前所未有的质量治理挑战。车规级软件作为关乎人身财产安全的强安全关键系统，对安全性、可靠性有着比较苛刻的行业壁垒与法规要求，必须严格遵循 ISO 安全标准^[3]及 ASPICE（汽车软件过程改进和能力评估）等行业过程模型^[4]。任何微小的软件缺陷若未能及时拦截，那到了量产阶段，都将面临极其高昂的亏损成本，甚至引发重大安全隐患。因此，构建一套高效、严密的软件项目全生命周期管理与项目质量监控系统，已成为当今汽车电子软件行业的迫切工程需求。

另一方面，在传统信息化建设模式下，企业内部各业务系统通常由不同部门独立建设，系统之间缺乏统一的数据协同机制，容易形成“数据孤岛”现象。这种模式不仅降低了数据共享效率，也增加了企业在运营分析、项目管理以及决策支持等方面的难度。随着企业数字化转型不断推进，企业对于运营效率、项目协同能力以及数据利用水平提出了更高要求，传统分散式的信息管理方式已经难以满足实际业务发展的需要。在此背景下，数字化运营平台逐渐成为企业数字化建设的重要组成部分^[5]。通过对企业运营数据、项目数据以及研发流程数据进行统一整合与集中管理，数字化运营平台能够实现数据资源的高效共享与协同利用，为企业运营管理、项目进度跟踪、资源调度以及经营决策提供数据支撑，从而提升企业整体管理效率和运营水平。

通过数字化运营平台的建设，企业能够有效打通不同业务系统之间的数据壁垒，缓解传统模式下数据分散、协同效率低等问题，进一步提升部门之间的信息共享与业务协同能力。同时，平台还能够为企业运营管理、项目监控以及资源配置提供更加及时、准确的数据支持，帮助管理人员更好地掌握项目进展和企业运行状态，提高决策的科学性与管理效率，从而增强企业整体运营能力和市场竞争力。

1.2 国内外研究现状

现代项目管理方法体系的形成与大型工程项目、国防工程和信息系统建设密切相关。随着项目规模不断扩大，传统依靠人工经验进行计划安排和过程控制的管理方式逐渐难以满足复杂项目的管理需求，项目管理开始向科学化、规范化和体系化方向发展。国外项目管理理论较早形成了较为成熟的方法体系，逐步将项目范围、进度、成本、质量、风险、

沟通和资源等内容纳入统一管理框架^[6]。随着项目管理专业化程度不断提高，项目经理逐渐成为企业组织中的重要管理角色，项目管理办公室也逐步承担起项目规范建设、过程监督、资源协调和决策支持等职能。

20 世纪 90 年代以后，随着计算机技术、网络技术和数据库技术的发展，项目管理软件系统逐渐成熟，并被广泛应用于工程建设、软件研发、企业运营和组织管理等领域^[7]。国外较有代表性的项目管理软件包括 Jira、Asana 等^[8-9]，这些系统通常围绕计划编制、任务分配、进度跟踪、缺陷管理、版本管理和团队协作等功能展开。随着敏捷开发、DevOps 和数字化运营理念的发展，国外项目管理系统逐渐由单一的计划管理工具向协同化、平台化和数据驱动方向演进，越来越重视项目过程数据的采集、分析和可视化展示，以支持管理者进行项目监控和决策分析^[10]。

国内项目管理信息化建设起步相对较晚，但伴随企业信息化和互联网行业的发展，项目管理软件系统也得到了较快发展。国内常见的项目管理和研发协作工具包括禅道、腾讯的 TAPD 等^[11-12]，这些系统在任务协作、需求管理、缺陷跟踪、文档管理、版本发布和项目统计等方面提供了较为完整的功能支持。与此同时，企业微信、钉钉、飞书等协同办公平台的普及，也推动了企业管理系统向在线化、协同化和轻量化方向发展。对于企业而言，项目管理系统已经不再只是项目经理记录任务和进度的工具，而是逐渐成为连接组织、人员、项目、流程和数据的重要管理平台。

从系统架构角度看，主流项目管理系统大致可以分为 C/S 架构和 B/S 架构两类^[13]。C/S 架构即客户端/服务器架构，通常具有响应速度较快、客户端能力较强和安全性较高等优点，但在客户端安装、版本升级和后期维护方面成本较高。B/S 架构即浏览器/服务器架构，用户通过浏览器即可访问系统，不需要单独安装客户端，具有部署方便、维护成本低、跨平台能力强等特点。随着网络基础设施不断完善以及 Web 前端技术的快速发展，B/S 架构在企业管理系统、项目管理系统和数字化运营平台中的应用越来越广泛，逐渐成为企业内部管理平台建设的主要形式。

从功能设计角度看，当前项目管理系统大致可以分为两类。一类是以研发项目管理为核心的专业化系统，如 Jira、禅道等，这类系统通常围绕产品、需求、项目、测试、缺陷、版本和发布等场景进行设计，功能较为完整，能够满足软件开发团队较复杂的过程管理需求，但由于功能模块较多，业务流程相对细致，用户学习成本和系统配置成本也相对较高。另一类是面向企业通用协作和数字化管理的平台型系统，这类系统通常通过字段配置、流程配置、权限配置、表单配置和数据看板等方式适配不同业务场景，具有较强的灵活性和复用性，适合企业根据自身管理流程进行定制化使用。基于 B/S 架构，简单易上手的软件项目管理系统必将是未来发展的方向。

综上，现有项目管理与研发协作系统在任务协同、缺陷跟踪方面已经较为成熟，但在

企业特定业务流程适配、缺陷质量数据统一度量以及轻量化使用方面仍存在不足。因此，面向汽车电子软件研发项目管理场景，设计并实现基于 B/S 架构的数字化运营平台，具有较强的实际应用价值。

1.3 论文的主要内容

本文的主要内容是对企业的数字化运营平台进行升级开发，围绕企业项目管理中缺陷数据来源分散、缺乏统一度量视图等问题，结合实际业务流程，对平台中的公司运营管理和项目运营管理相关功能进行分析、设计、实现与测试。论文对系统开发涉及到的技术、功能的需求规格、系统设计、系统开发与测试工作做了介绍。

本系统采用 B/S 架构和前后端分离模式。在系统技术选型上，后端选用 Spring Boot 框架构建业务服务，前端基于 Vue 实现交互界面。在数字化运营平台使用过程中，一个直观全面的统计报表能让用户快速获取到关键信息，再加上报表的使用场景广泛，因此本系统还引入了 Echarts 来满足可视化报表的需求。并介绍了数据库 MySQL，Redis 的相关原理，将其和同类型的产品做了横向对比来体现其优势所在。

在系统功能上，本文重点围绕部门投入 BU 工时分布、项目基本信息管理、缺陷管理和重大问题管理四个功能模块，完成了从需求分析、详细设计到系统实现与测试的完整开发过程，并针对缺陷深分页查询和 BU 工时统计查询中出现的性能问题进行了优化，以提升系统在实际使用场景下的响应效率。

1.4 论文的组织结构

本文共分为七章，各章内容安排如下：

第一章为绪论。介绍了课题的研究背景与意义，梳理了国内外在项目管理系统、研发效能度量及相关技术领域的研究现状，并给出了本文的主要内容。

第二章为相关开发技术介绍。对系统开发中使用的 SpringBoot、Vue、MySQL、Redis、ECharts 和 MyBatis-Plus 等相关开发技术进行了说明。为后续的系统设计与实现提供技术基础。

第三章为需求分析。围绕部门经理、项目经理、项目成员、质量人员这几个角色，对部门投入 BU 工时分布、项目基本信息管理、缺陷管理和重大问题管理四个模块进行了功能需求分析，并补充了性能与安全性方面的非功能性需求。

第四章为系统总体设计。采用前后端分离架构，将系统按照分层架构划分为前端展示层、请求处理层、业务逻辑层、数据访问层和数据层五个层次，并从业务维度将系统横向划分为系统管理、公司运营管理和项目运营管理三个大模块。

第五章为核心业务模块的详细设计与实现。对部门投入 BU 工时分布、项目基本信息管理、缺陷管理和重大问题管理四个核心模块进行了详细设计，结合类图与时序图描述了

各模块的接口依赖和业务逻辑过程，并给出了数据库的 E-R 模型与关键表结构设计。

第六章为系统测试。首先介绍了系统测试环境，然后通过功能性测试验证了各业务模块的逻辑正确，并用图片展示了各模块的实现效果，同时对系统的非功能性指标进行了评估。

第七章为总结与展望。总结了本文的研究工作，分析了系统当前的不足，提出了后续改进方向。

第二章 相关开发技术介绍

为了提高系统的开发效率与可维护性，本文在系统开发过程中采用了当前较为成熟的企业级开发技术与相关框架。这些技术能够高效支持系统的功能实现、数据管理以及前后端交互，同时也有助于提升系统的稳定性与扩展性。下面将对本系统开发过程中使用的主要技术进行介绍。

2.1 SpringBoot 框架

在企业级系统开发中，稳定性是需要优先考虑的因素。Java 自 1995 年问世以来，经过多年发展，已经在企业应用领域形成了较成熟的运行机制。依托 JVM，Java 具备较好的跨平台能力^[14]，能够实现“一次编写，到处运行”，便于系统在不同环境中的部署和迁移，常用于开发 Web 应用程序。企业开发同样重视技术生态。Java 拥有较完善的框架和工具链。在众多 Java 后端开发框架中，SpringBoot 以快速开发和生态丰富的优势占据了主要地位。下面从其核心理念与关键特性两个方面进行介绍。

2.1.1 SpringBoot 框架核心理念

在 Java 企业级应用开发中，Spring、Spring MVC 与 MyBatis 等传统架构需要维护较多的 XML 配置文件，手工管理依赖版本并执行多步骤部署流程。启动新项目时，开发者需投入相当一部分时间处理框架整合与组件装配，而非直接编写业务代码。Spring Boot 的设计目标正是减少这一部分非功能性的搭建开销。

Spring Boot 是 Spring 生态的扩展，其核心机制为“约定优于配置”^[15]：框架为常见场景预设了默认参数，开发者无需从零编写样板配置即可启动应用。在日常开发中，通过注解即可完成 Bean 的注册与依赖装配，省去了显式的 XML 声明或工厂方法。这一方式使开发者能将精力集中于业务逻辑，而非底层环境搭建，降低了初始配置的复杂度。

2.1.2 SpringBoot 框架关键特性

Spring Boot 相较于传统 SSM 架构，主要在以下四个方面做了简化：

- (1) 控制反转与依赖注入：Spring Boot 延续了 Spring 的 IoC 容器机制。对象的创建与生命周期由容器统一管理，不再由开发者在代码中实例化，开发者通过注解声明依赖关系，容器在运行时自动注入关联实例。这一机制降低了模块间的编译期耦合，也便于单元测试时替换桩实现，同时还减少了冗余代码量。
- (2) 起步依赖：框架将特定功能场景所需的依赖库封装为标准化的 Starter^[16]。开发者在 pom.xml 中引入对应 Starter 后，Maven 自动解析并下载所有传递性依赖，避免了手动排查版本冲突的问题。
- (3) 外部化配置：系统在开发、测试与生产环境中通常需要不同的数据库连接、中间件地

址和第三方密钥。Spring Boot 支持通过 `application.yml` 为不同环境预设独立的配置文件^[17]，并在启动时通过命令行参数、环境变量或配置中心指定激活的配置集，无需重新编译即可切换运行环境。

- (4) 内嵌式运行容器：传统 Java Web 应用需先部署到外部 Tomcat 服务器再启动。Spring Boot 直接将 Tomcat 容器集成到项目中，通过 `mvn package` 打包为独立可执行 Jar 包^[18]，使用 `java -jar` 命令即可启动服务，减少了独立部署与运维配置的步骤。

首行缩进

Spring Boot 的自动装配与内嵌容器机制能够减少项目初始化、依赖配置和部署配置中的重复工作，IoC 容器与起步依赖有助于降低模块间的耦合度。结合本系统在开发过程中需要频繁迭代功能、快速验证业务逻辑的实际需求，本文选择 Spring Boot 作为后端开发框架。

2.2 Vue 框架

引用放到这里

Vue 是一个轻量级前端框架，采用“渐进式”设计^[19]。开发者既可以将其用于页面局部功能增强，也可以结合路由、状态管理和构建工具等生态组件构建单页面应用（SPA）。与直接操作 DOM 的传统开发方式相比，Vue2 在底层遵循 MVVM（Model-View-ViewModel）架构模式，如图2-1所示。

加个参考文献

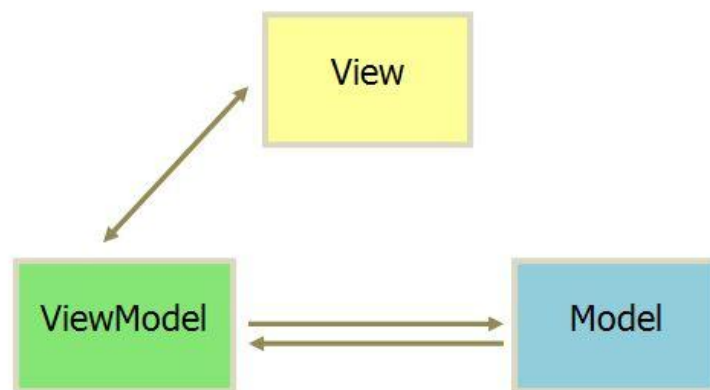


图 2-1 MVVM 模式图

在 MVVM 架构中，Model 层存储数据状态与业务逻辑；View 层对应页面呈现；ViewModel 层则承担二者之间的数据绑定与交互协调作用。当 Model 数据变化时，View 可根据绑定关系自动更新；当用户与 View 交互时，Model 的数据同步被修改。这种双向绑定省去了手动操作 DOM 的步骤，开发者可以专注于业务数据的流转。

与 React、Angular 等其他常用前端框架相比，Vue 更适合本系统的前端开发需求。React

具有较强的灵活性，适合复杂交互界面的开发，但工程结构通常需要开发团队自行组织；Angular 提供了较完整的工程体系和开发规范，但框架体量较大，学习和维护成本相对较高。Vue 的模板语法较为直观，组件化开发方式清晰，项目结构也相对轻量。结合本系统以表单录入、表格查询、条件筛选和统计展示为主的后台管理场景，Vue 能够在开发效率、页面维护性和后续扩展性之间取得较好平衡，因此本文前端部分选择 Vue 作为主要开发框架。

在本系统前端实现中，Vue 比较擅长页面组件组织、用户交互处理等工作，但对于图表绘制和可视化交互的场景支持有限。因此需要使用 Echarts 库来处理这部分工作，它是一款基于 JavaScript 的开源 Web 数据可视化库，支持折线图、柱状图、饼图等常见图表类型^[20]。

2.3 数据库技术

在企业级 Web 应用中，业务数据通常需要长期保存，并支持较为稳定的查询、统计和更新操作。因此，系统在数据存储层采用 MySQL 保存业务数据。同时，考虑到部分基础数据和统计结果会被前端页面反复访问，系统引入 Redis 作为缓存组件，减少重复查询数据库带来的开销，提高接口响应速度。

2.3.1 关系型数据库 MySQL

MySQL 是目前应用较为广泛的开源关系型数据库管理系统之一，具有部署成本低、使用门槛较低、社区资料丰富等特点，能够满足本系统对业务数据存储、条件查询和统计分析的需求。与 Oracle 相比，MySQL 不需要较高的授权和运维成本，更适合本系统这类企业内部管理平台的开发与部署；与 PostgreSQL 相比，MySQL 在国内企业应用和 Web 后端开发中使用更为普遍，相关工具、驱动和开发经验也更容易获取。因此，综合开发成本、维护便利性和系统实际需求，本文选择 MySQL 作为系统的关系型数据库。其运行机制如图 2-2 所示。

客户端通过 TCP 协议连上 MySQL 服务后，会先做权限校验和身份确认，这一步由连接器负责，为了提高速度和资源利用率，MySQL 会用连接池来进行连接。接着，系统会看看之前有没有执行过一模一样的 SQL（但是查询缓存在最新的 MySQL 版本中因为命中率太低被移除了）。之后这条查询语句会走到解析器那里，做词法和语法检查，看看语句写得对不对；然后再交给预处理器，验证里面涉及的表和字段是不是真的存在。再往后就是优化器登场，它会根据各种情况算一算哪种执行方案成本最低，比如决定用哪个索引来查。最后执行器按这个计划去调用存储引擎提供的接口，把符合条件的记录一条条取出来，最终将结果集打包返回给客户端。

本系统选用 MySQL 默认的 InnoDB 存储引擎。InnoDB 支持事务处理，能够较好地保

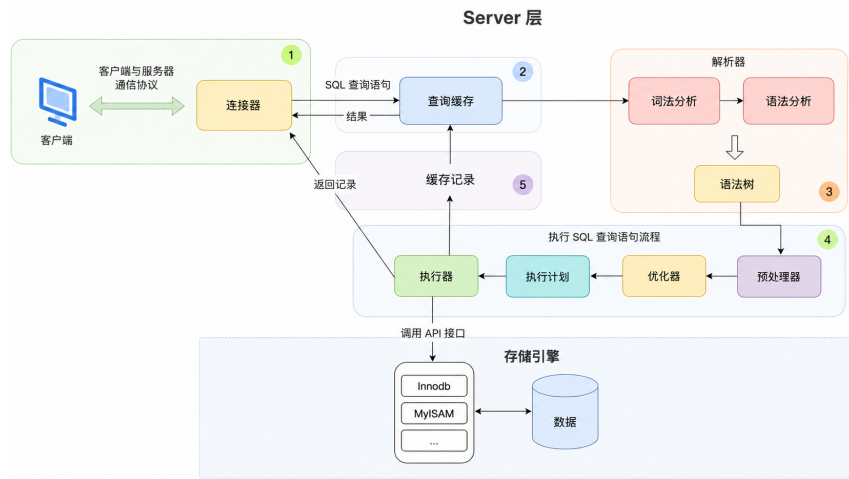


图 2-2 MySQL 执行流程图

证业务数据在修改过程中的一致性。其索引底层通常采用 B+ 树结构，有利于减少磁盘 I/O，提高条件查询效率。同时，InnoDB 还支持 MVCC（多版本并发控制）和行级锁机制，在多用户同时访问系统时，可以在一定程度上降低读写冲突对业务操作的影响，保证并发环境下的数据隔离性和可靠性^[21-22]。

2.3.2 非关系型数据库 Redis

MySQL 能够提供稳定可靠的数据持久化服务，但在面对访问频率较高且数据结构相对简单的场景，如果每次都直接访问数据库，会增加数据库压力，也会影响接口响应速度。因此，本系统引入了 Redis 作为缓存中间件。Redis 本质上是一款由 C 语言实现的，基于内存的高性能键值对存储系统，由于其数据读写操作主要在内存中进行，因此具有较低的访问延迟。除了基础的字符串类型，Redis 还原生支持哈希、列表、集合、有序集合等丰富的数据类型，开发者能够很便捷地将其应用于排行榜，分布式锁，打卡签到等场景^[23]。为了兼顾数据安全性，Redis 提供了 RDB 快照和 AOF 日志两种持久化方式，前者定期保存全量数据，后者以追加形式记录每条写命令，两者可组合使用以平衡恢复速度与数据完整性^[24]。在性能方面，Redis 采用单线程模型处理命令，避免了多线程的锁竞争和上下文切换开销，同时借助 I/O 多路复用技术让单个主线程能够并发处理大量客户端连接。基于以上特性，在流量较大或者数据库查询比较慢的接口，引入 Redis 可以有效减少到达数据库的请求数量，提高系统的性能和稳定性。

在缓存组件选型时，本系统没有采用 Memcached，原因在于 Memcached 更适合简单的键值缓存场景，而本系统后续可能涉及对象缓存、权限信息缓存以及统计结果缓存等多种数据形式。Redis 在数据结构、持久化能力和功能扩展方面更符合系统需求，并且支持通过 Lua 脚本将多个操作组合为原子执行^[25]。因此，综合系统功能扩展、数据结构适配和后续维护成本，Redis 更适合作为本系统的缓存中间件。

2.4 MyBatis-Plus

MyBatis-Plus 是在 MyBatis 基础上扩展形成的持久层开发工具，其设计理念是“只做增强不做改变”，即在保留 MyBatis 原有使用方式的基础上，进一步简化常见数据库操作^[26]。在后台管理类系统中，数据访问层通常包含大量结构相似的增删改查代码，如果全部手动编写 Mapper 方法和 SQL 语句，容易造成代码重复，也会增加后期维护成本。因此，本文在持久层开发中引入 MyBatis-Plus，用于简化基础数据访问逻辑。

在具体使用时，持久层接口可以通过继承 BaseMapper 获得常用的单表增删改查方法，开发者只需定义实体类与数据库表之间的映射关系，便可完成大部分基础数据操作。对于带有不固定查询条件的接口，MyBatis-Plus 提供了 QueryWrapper、LambdaQueryWrapper 等条件构造器，可以根据前端传入的参数动态生成查询条件，避免在代码中手动拼接 SQL 字符串。对于列表类页面中常见的分页查询需求，MyBatis-Plus 也提供了分页支持，能够配合页码、每页条数等参数返回对应范围的数据结果。

MyBatis-Plus 并不会限制原有 MyBatis 的使用方式。对于单表操作和简单条件查询，可以使用框架提供的通用方法完成；对于多表关联、复杂统计等场景，仍然可以在 Mapper XML 文件中编写自定义 SQL。通过这种方式，系统既能够减少基础持久层代码的重复编写，又能保留复杂查询场景下的灵活性。

2.5 本章小结

本章介绍了系统开发中使用的几项技术：后端采用 Spring Boot 框架，借助自动装配和内嵌容器机制简化服务端开发与部署；前端选用 Vue 框架，通过 MVVM 模式实现数据与视图之间的联动，降低页面交互开发的复杂度；数据层采用 MySQL 与 Redis 组合，前者负责业务数据的持久化存储，后者用于缓存部分访问频率较高的数据；持久层使用 MyBatis-Plus 减少基础 CRUD 代码的重复编写；数据可视化部分则通过 ECharts 完成图表展示。上述技术为后续系统需求分析、功能设计与实现提供了基础支撑。

第三章 需求分析

本章围绕企业数字化运营平台的建设需求展开分析。首先，对系统应具备的总体功能进行概述，使读者能够从整体上了解系统建设的背景和意义；随后，进一步从功能需求和非功能性需求两个方面进行分析。其中，功能需求主要描述系统各业务模块应实现的具体功能，是后续系统架构设计、模块划分和业务流程设计的重要依据。非功能性需求则主要关注系统在实际运行过程中的稳定性、安全性、可扩展性和易用性等方面，既要保证系统能够长期稳定运行，也要提升用户在使用过程中的流畅性、便捷性和可靠性。

3.1 需求概述

本系统是某汽车电子公司内部的数字化运营管理系统，面向企业内部的软件项目管理和运营管理场景，主要用于解决项目过程数据分散、统计分析依赖人工整理、业务流程跟踪不能自动化等问题。系统需要将项目、组织、人员、工时、质量和运营指标等数据进行整合，为管理人员提供较为统一的数据查看和业务处理平台。

系统的使用对象包括项目经理、部门经理、项目成员（开发测试人员）以及质量人员等。不同角色在系统中的关注内容和操作权限不同，因此系统需要具备统一的身份认证、权限控制 and 数据访问管理能力，以保证业务数据的安全性。本系统的建设目标是将项目管理、运营分析和系统基础管理整合到同一平台中，减少分散系统和人工统计带来的管理成本，提高企业内部项目过程管理和运营数据分析的效率。

从业务范围上看，系统主要包括三个部分。第一部分是项目管理，围绕项目基础信息、版本范围、质量问题和风险事项等内容，对项目执行过程进行记录、查询和跟踪。第二部分是公司运营管理，主要面向组织、员工、工时和 KPI 等运营数据，支持从部门、人员、项目等角度进行统计分析。第三部分是系统管理，该部分主要作为系统基础支撑，提供用户、角色、菜单、部门、权限和日志等后台管理功能，为业务模块的正常运行提供支撑。

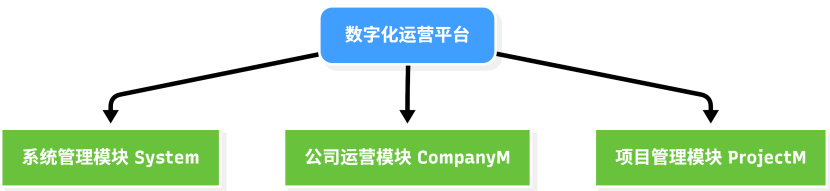


图 3-1 玄武平台模块图

3.2 功能需求

本文的实现范围主要包括公司运营管理模块中的部门投入 BU 工时统计功能，以及项目管理模块中的项目基本信息管理、缺陷管理和重大问题管理功能。

在相关功能中，部门经理主要关注部门工时在不同 BU 方向上的投入情况；项目经理

主要负责项目基础信息维护和项目过程跟踪；项目成员（开发测试人员）参与缺陷和重大问题的处理与反馈；质量人员则重点关注缺陷数据、重大问题状态以及相关质量风险。为便于后续系统设计，下面将按照功能模块分别分析各功能的参与角色、业务目标和主要操作需求。

3.2.1 部门投入 BU 工时分布需求分析

BU 工时投入分析模块旨在为部门经理提供下属员工在各 BU（Business Unit）上工时分布的全景视图，为人力资源调配与项目成本核算提供数据支撑。该模块仅涉及部门经理一类参与者。如图3-2所示，系统需要提供按年月和组织树两个维度的工时数据检索能力，支持以饼图形式呈现部门整体的 BU 工时分布情况。同时，系统需要支持对特定 BU 的工时数据进行详细分析，提供总工时、平均工时和加班工时等基础统计指标的计算能力，并能够生成近六个月的工时趋势图，以便识别工时投入的变化规律。

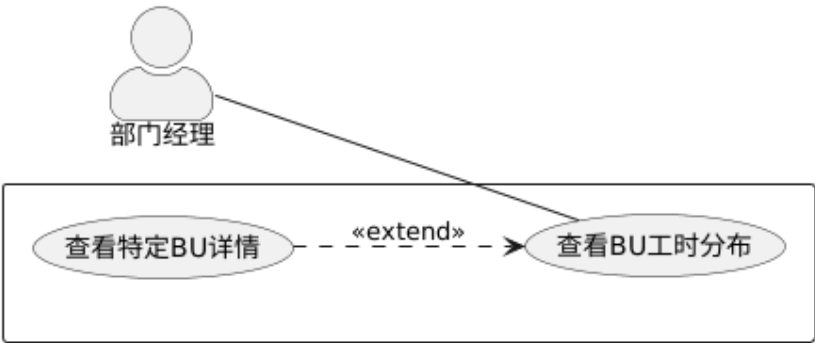


图 3-2 BU 工时分布用例图

表 3.1 部门投入 BU 工时统计用例描述

用例名称	参与者	用例描述
查看 BU 工时分布	部门经理	部门经理进入工时分析页面后，系统按照 BU 维度统计部门当月工时投入情况，并以饼图形式展示各 BU 的工时占比。
查看特定 BU 详情	部门经理	部门经理可在 BU 工时分布结果中选择某一 BU，进一步查看该 BU 下的工时指标和工时趋势。

3.2.2 项目基本信息管理需求分析

项目信息管理模块主要用于维护项目的基础资料和交付计划信息，围绕项目的基本属性和交付节点展开，既需要保证项目基础信息的完整性，也需要记录项目关键交付节点的变化情况。该模块主要涉及项目经理和项目成员两类参与者。项目经理是项目信息维护的主要责任人，具有新增项目、编辑项目基础信息、填写交付日期以及查看交付历史等权限；项目成员主要作为信息使用者，能够查询项目基本信息及历史交付记录。

如图 3-3 所示，项目信息管理模块可划分为项目基本信息管理和里程碑计划管理两个主要部分。项目基本信息管理主要包括项目信息的新增、编辑、查询和导出；里程碑计划

管理主要包括交付日期填写和交付历史查看，用于反映项目关键节点的计划与变更情况。

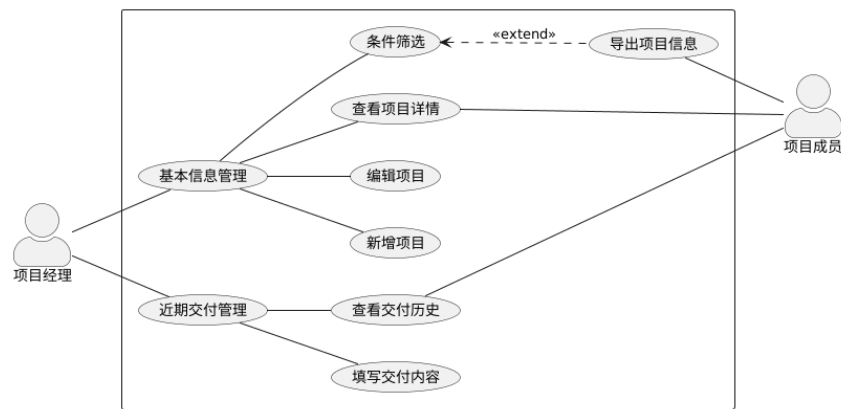


图 3-3 项目信息管理用例图

表 3.2 项目基本信息管理用例描述

用例名称	参与者	用例描述
条件筛选	项目经理、项目成员	用户可根据项目名称、项目所属 BU、时间范围等条件筛选项目数据。
查看项目详情	项目经理、项目成员	用户可查看项目的详细信息。
新增项目	项目经理	项目经理可新增项目基础信息。
编辑项目	项目经理	项目经理可对已有项目基础信息进行修改。
填写交付节点	项目经理	项目经理填写项目当前阶段最关注的一个近期交付事项。
查看交付历史	项目经理、项目成员	用户可查看项目交付的修改历史。
导出项目信息	项目经理、项目成员	用户可将筛选后的结果导出为 Excel 文件。

3.2.3 缺陷管理需求分析

缺陷管理模块主要用于对项目缺陷数据进行集中查询和统计分析，帮助质量人员了解项目当前的缺陷分布和质量状态。该模块的数据主要面向项目质量跟踪场景，既支持缺陷明细查看，也支持从项目聚类角度观察多个项目的缺陷健康情况。

如图 3-4 所示，缺陷管理模块主要涉及质量人员和项目成员两类参与者。质量人员可以查看缺陷列表，并根据需要将缺陷数据导出为 Excel，便于后续分析和归档。同时，质量人员还可以查看缺陷健康概览，对项目聚类进行维护，包括新增、编辑和删除聚类信息，并进一步查看聚类下包含的项目及其缺陷明细。当系统发现项目缺陷情况需要关注时，质量人员可以发送缺陷预警，项目成员则作为提醒接收方，接收相应的缺陷提醒信息。

表 3.3 缺陷管理用例描述

格式

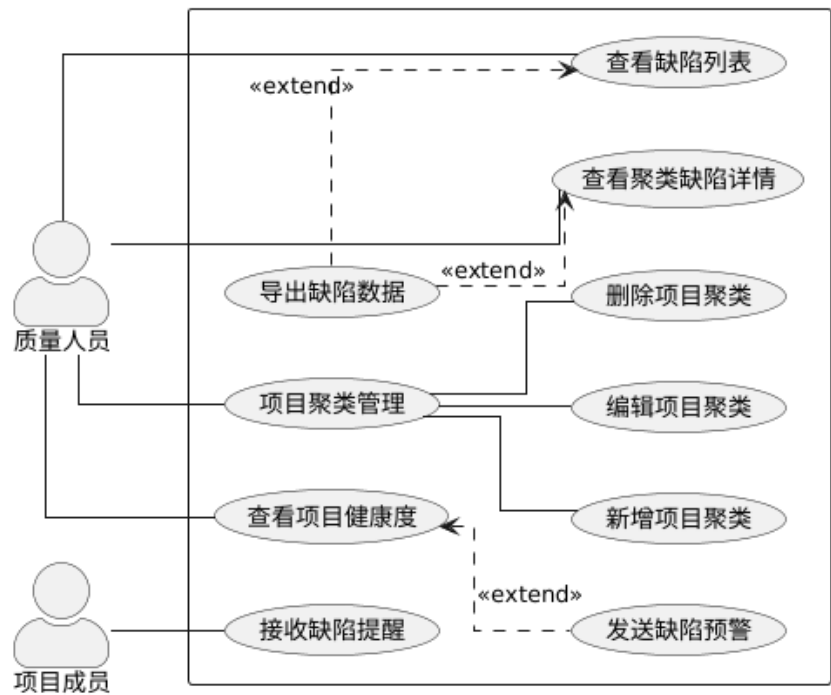


图 3-4 缺陷管理用例图

用例名称	参与者	用例描述
查看缺陷列表	质量人员	查询数据库中的缺陷记录，支持条件筛选。
查看项目健康度	质量人员	展示各项目聚类的整体健康状态, 可手动向项目成员发出预警提醒。
接收缺陷提醒	项目成员	在企业内部沟通平台接收机器人推送的缺陷预警通知。
导出缺陷数据	质量人员	查看缺陷列表或聚类缺陷详情时, 可按需导出为 Excel 文件。
查看聚类缺陷详情	质量人员	查看某一项目聚类下的具体缺陷明细。
新增项目聚类	质量人员	创建一个新的项目聚类，可以纳入多个项目。
编辑项目聚类	质量人员	修改已有项目聚类的基本信息。
删除项目聚类	质量人员	删除不再使用的项目聚类。

3.2.4 重大问题管理需求分析

重大问题管理模块主要用于记录和跟踪项目执行过程中影响范围较大、处理周期较长或需要重点关注的问题。该模块不仅需要保存问题的基本信息和处理状态，还需要支持问题查询、详情查看、统计分析和消息通知，以便质量人员及时掌握问题进展。

如图 3-5 所示，重大问题管理模块主要涉及质量人员和项目成员两类参与者。质量人员是该模块的主要使用者，可以查询重大问题、查看问题详情、新增和编辑问题记录，并查看重大问题的分布情况和变化趋势。对于查询结果，系统还需要支持问题数据导出。当新增重大问题时，系统需要向相关人员发送通知，项目成员则作为通知接收方，及时了解

与自身项目或职责相关的重大问题信息。

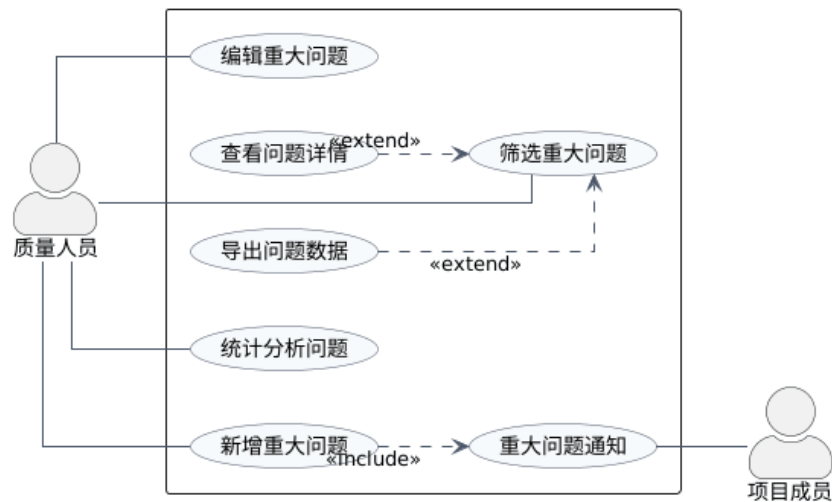


图 3-5 重大问题用例图

表 3.4 重大问题管理用例描述

用例名称	参与者	用例描述
筛选重大问题	质量人员	按条件检索重大问题列表。
查看问题详情	质量人员	从筛选结果中点选一条记录，查看完整信息。
新增重大问题	质量人员	录入一条新重大问题，同时自动发送通知。
编辑重大问题	质量人员	修改已录入重大问题的内容。
导出问题数据	质量人员	将筛选得到的问题列表导出为文件。
统计分析问题	质量人员	对重大问题做统计和可视化分析。
重大问题通知	质量人员、项目成员	新增重大问题时，系统会通过企业交流平台自动向项目成员推送通知。

3.3 非功能需求

在企业实际使用场景中，系统还需在性能、可靠性和易用性等方面满足一定的约束，以维持平台在数据量增长后仍能正常运行。

3.3.1 性能要求

按照汽车软件研发团队的规模推算，系统上线后工时流水记录和缺陷表的数据量会在短时间内快速增长，单张数据表的数据量会达到数十万级别。这对涉及多表关联和分组聚合的统计查询提出了较高要求，相关接口需要在大数据量下保持合理的响应速度。特别是前端首页通过 ECharts 渲染的 BU 工时看板和项目健康度图表，如果数据接口响应过慢导致页面长时间空白或请求超时，会直接影响用户的工作效率。

3.3.2 接口可靠性

系统在调用飞书接口发送提醒消息时，可能会遇到网络超时、接口限流或第三方服务异常等情况，因此后端需要对调用过程进行异常捕获和日志记录，并在连续失败时提醒管理员处理。对于系统自身接口，在入口处应先对前端参数进行校验，参数缺失或格式不正

确时直接返回错误结果，避免无效请求进入业务层。业务层在使用 Redis 缓存时，也需要考虑缓存穿透、击穿和雪崩等问题，减少异常请求或热点数据失效时对数据库的冲击。数据访问层编写动态 SQL 时，应对可选参数进行非空判断，只在参数有效时拼接查询条件，避免因空参数造成 SQL 语法错误。通过这些处理，可以提高接口在异常输入、外部接口失败和高并发访问情况下的稳定性。

3.3.3 易用性

系统在设计时需要尽量降低用户的操作成本，使用户能够较快理解各功能模块的入口和操作方式。在页面设计上，系统应保持统一的布局风格和交互方式。常用功能应放置在用户容易发现的位置，例如查询、重置、新增、导出等操作应在列表页面中保持相对固定的位置，减少用户在不同页面之间切换时的学习成本。对于表单填写类功能，系统应提供明确的字段名称和必要的校验提示，避免用户因字段含义不清或输入格式错误而反复修改。对一些提交按钮，前端要做置灰处理以防止用户的连续点击。操作完成后，系统应对常见操作结果给予及时反馈。例如数据保存成功、导出完成、查询无结果或操作失败时，应给出清晰提示，帮助用户判断操作是否生效以及下一步应如何处理。

3.4 本章小结

本章对系统的需求进行了分析。在总体层面，明确了系统面向项目经理、部门经理、项目成员和质量人员四类用户，涵盖项目管理、运营管理和系统管理三个业务领域。在功能层面，围绕本文的实现范围，分别对部门 BU 工时统计、项目信息管理、缺陷管理和重大问题管理四个模块的业务目标、参与角色和主要操作需求进行了说明。在非功能层面，从性能、可靠性和易用性三个方面提出了系统上线后需要满足的基本约束。

第四章 系统总体设计

数字化运营平台面向企业内部多个部门的项目管理人员和运营团队，使用人员分布在研发、测试、质量和管理等多个岗位，日常办公终端涵盖 Windows 台式机和笔记本等多种设备，终端环境不统一。平台在上线后需要根据业务变化持续迭代功能，对版本发布和升级的便捷性有一定要求。如果采用 C/S 架构，客户端的安装、升级和版本维护需要逐一适配不同终端环境，会增加后期运维成本。综合上述使用场景和维护需求，本文采用 B/S 架构进行设计与开发，用户只需通过浏览器即可访问系统，能够降低终端适配和版本管理的复杂度。

4.1 系统整体架构设计

为了降低模块之间的耦合度，并提高系统后续维护和扩展的便利性，本文在整体架构设计中采用分层设计思想，将系统划分为前端展示层、请求处理层、业务逻辑层、数据访问层和数据层五个部分^[13]。各层之间按照相对明确的调用关系进行协作，上层主要负责接收用户请求和展示处理结果，下层则负责业务处理、数据访问和数据持久化。

具体而言，表示层由前端页面组成，负责用户交互、数据展示和请求发送；业务逻辑层负责处理项目管理、公司运营管理等核心业务规则；数据访问层负责封装对数据库的访问操作；数据存储层则用于保存系统运行过程中产生和同步的业务数据。通过这种分层方式，系统的页面展示、业务处理和数据操作能够保持相对独立，有利于提高系统结构的清晰度。系统整体架构如图 4-1 所示。

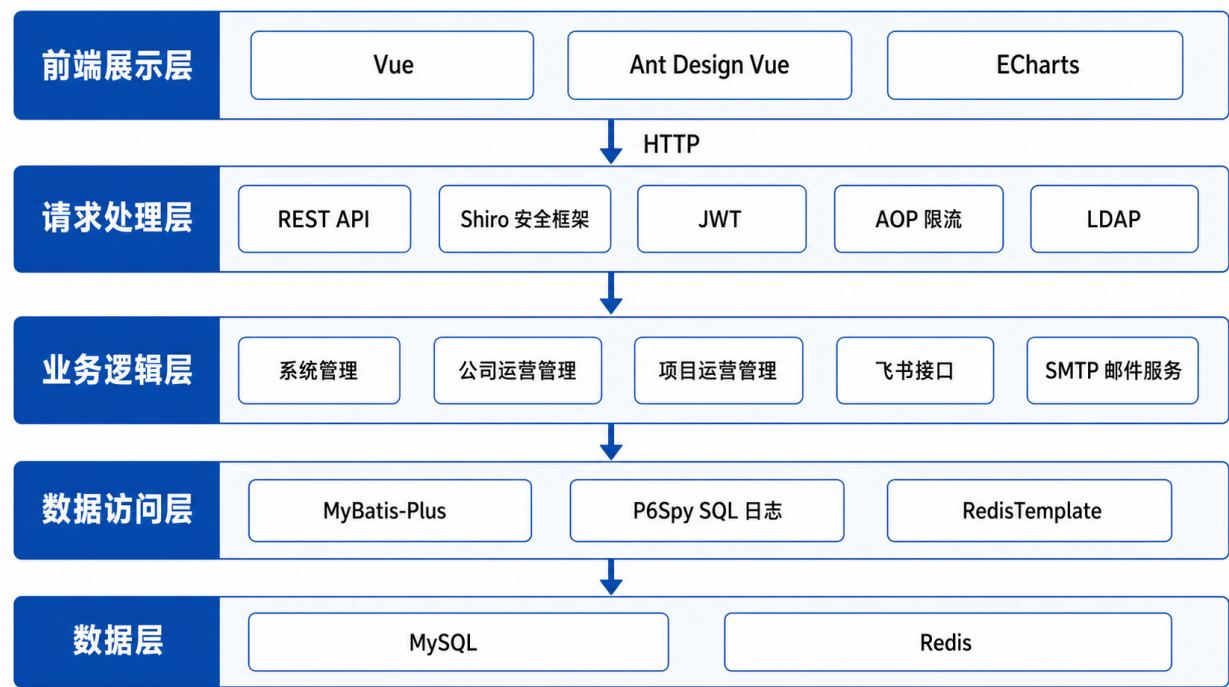


图 4-1 系统架构图

箭头删除

第一层为前端展示层。负责系统页面展示、用户交互和数据可视化等工作。本系统前端基于 Vue 进行开发，页面组件和基础交互控件主要采用 Ant Design Vue 实现，能够满足表单、表格、按钮、弹窗等后台管理系统常见界面的开发需求。对于数据展示场景，系统使用 ECharts 绘制柱状图、折线图、饼图等可视化图表，使用户能够更直观地理解统计结果。前端通过 Axios 与后端接口进行 HTTP 通信，然后从后端返回的 JSON 格式中拿到数据。前端项目运行环境依赖 Node.js，并通过 npm 管理前端依赖，使用 Webpack 完成项目打包和资源构建。

第二层为请求处理层。位于前端界面和后端业务处理之间，主要负责身份认证、权限校验和接口访问控制，作为后端服务的统一“过滤器”。本系统使用 Shiro 安全框架实现用户认证与授权管理，通过 JWT 在用户登录后颁发身份令牌，前端在访问登录接口后拿到令牌并保存到本地，后续所有请求都在请求头中携带该令牌，后端对令牌有效性和用户权限进行校验。用户登录时，系统结合 LDAP 进行身份认证，以便与企业内部账号体系保持一致。除身份和权限控制外，系统还通过 AOP 自定义注解方式对部分接口进行限流处理，避免短时间内重复请求对系统造成过大压力。

第三层为业务逻辑层。该层是系统后端的核心部分，主要负责项目管理、公司运营管理等业务功能的处理。前端发起的 HTTP 请求在通过安全校验后，会根据请求方式和请求路径分发到对应的后端接口，并进入具体的业务模块。在业务处理过程中，系统首先对请求参数进行必要的校验，保证输入数据符合接口处理要求。对于部分统计类或查询频率较高的数据，业务逻辑层会优先判断缓存中是否存在有效结果；若缓存命中，则直接返回缓存数据，减少重复计算和数据库访问；若缓存中不存在有效数据，则继续执行相应的业务规则处理、数据统计计算和结果封装等操作。业务逻辑层在需要访问数据库时，会调用数据访问层提供的数据库访问方法。最终，业务逻辑层将处理结果封装为统一的响应对象，并返回给前端展示。

第四层为数据访问层。数据访问层负责屏蔽底层数据读写细节。MyBatis-Plus 提供基础的数据访问能力，但是由于过度封装，相比于传统的 XML 手写 SQL 来说并不是很直观。为了满足查看 SQL 执行计划的需要，使用 P6Spy SQL 日志用于记录和分析 SQL 执行情况，RedisTemplate 用于操作缓存数据。业务逻辑层不直接访问数据库，而是通过这一层提供的接口完成数据查询和写入。

第五层为数据层。数据层负责系统数据的持久化和缓存支撑。MySQL 存储系统权限、用户、项目、缺陷、运营等核心业务数据；Redis 用于缓存、会话和高频访问数据，提升热点数据查询的响应速度并减轻数据库压力。

4.2 系统功能模块划分

在上述分层架构的基础上，从业务维度对后端工程进行横向模块划分，将系统拆分为三个核心业务域：系统管理、公司运营管理和项目运营管理，以降低模块间的耦合度并提升代码的可维护性。从整体功能上看，项目管理模块主要围绕汽车电子软件过程管理展开，包括项目基础信息、缺陷、重大问题、风险、版本范围等功能；公司运营管理模块主要面向企业组织、员工、工时和 KPI 等运营数据，支持企业内部运营情况的统计分析；系统管理模块主要包括用户管理、角色管理、菜单管理、部门管理、权限管理和日志管理等功能，用于保障系统的正常运行。

其中，项目管理和公司运营管理承担主要业务功能，系统管理为业务模块提供用户、权限和菜单等基础支撑。本文根据毕业设计任务分工，重点对其中部分功能模块进行设计与实现，包括公司运营管理下的部门投入 BU 工时分布，项目运营管理下的项目基本信息管理、缺陷管理和重大问题管理，如图4-2所示。

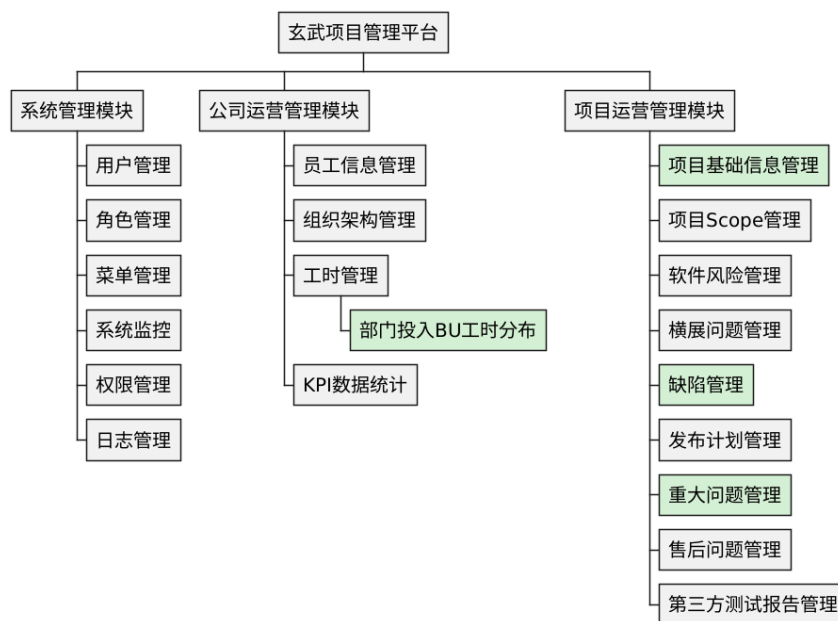


图 4-2 系统功能模块划分图

(1) 部门投入 BU 工时分布

主要用于从部门维度统计各业务单元的工时投入情况。该功能面向管理人员对部门资源投入结构的分析需求，通过对工时数据、员工信息和组织信息进行关联汇总，展示指定部门在不同 BU 方向上的工时分布情况，从而帮助管理人员了解部门人力资源在各业务方向上的投入比例和变化趋势。该功能是部门经理使用的，前端每次发送的请求都会先经过请求处理层看看用户携带的 Token 是否有效，之后会解析出用户的 ID, 然后通过系统管理模块下的权限校验功能去数据库中查询该用户是否具备权限。

(2) 项目基本信息管理

项目基本信息管理主要面向项目经理使用，用于对项目基础数据进行查询和维护，内容包括项目名称、项目类型、所属 BU、里程碑计划、平台、客户以及项目状态等信息。对于项目信息在运行过程中的变更，系统会保留相应的历史记录，从而保证项目数据的可追溯性。该模块也为其他业务功能提供基础数据支撑。例如，在 BU 工时分布中，为满足数据库范式要求，工时数据不会记录项目属于哪个业务单元，还需要关联项目信息表来查询。在新增风险、重大问题时，表单中的下拉框数据也需要使用该模块的接口来查询。

(3) 缺陷管理

缺陷管理模块主要用于对项目缺陷数据进行查询、统计和导出，帮助质量人员掌握项目质量状态。这些缺陷数据来自与系统使用的 Trinity Bug、Jira 这三个缺陷管理平台，会通过定时任务同步到本地的数据库中。用户可以查询对缺陷数据进行分页查询，也可以将多个项目看作一个聚类来进行汇总分析。对于健康度较低的项目聚类，质量人员可以发送飞书消息提醒，这里就需要通过组织架构管理和员工信息管理模块的接口来确认消息的接收人员列表。此外，一个缺陷被发现并修复后，可能也会在其他项目中暴露，项目经理可以标记缺陷为需要横展，之后进入横展问题管理模块的流程。

(4) 重大问题管理

重大问题管理模块主要用于对项目管理中比较严重的问题进行跟踪和处理。质量人员可以在该模块中维护重大问题的基本信息、责任人、所属项目、客户及处理进展等内容。对于新增或需要重点关注的问题，系统支持通过飞书消息进行通知，使相关责任人能够及时接收到问题信息并开展处理。重大问题可能由风险转入，也可能由横展引发。对一个重大问题，项目经理同样可以标记为需要横展，进入横展的流程。

4.3 本章小结

本章主要对玄武数字化项目管理平台进行了总体规划与设计。首先从分层角度阐述了系统的五层架构，包含前端展示层、请求处理层、业务逻辑层、数据访问层与数据层，并标注了各层的核心技术栈选型。随后，从实际业务维度出发，将系统横向解耦为系统管理、公司运营管理和项目运营管理三大核心模块，并简要介绍了本文负责的功能模块主要职责。

第五章 核心业务模块的详细设计与实现

上一个章节从整体架构、功能模块等层面对系统进行了宏观设计，明确了各模块的职责。在此基础上，本章将对系统的核心模块展开详细设计，将概要设计转化为可落地代码方案。本章从系统的各个模块和几个关键点设计出发，结合类图与时序图来解释类之间的关系以及核心方法的业务逻辑，为后续的系统实现与测试提供关键性的保障。

5.1 模块设计

这个系统所有模块在工程上都采用了标准的 Controller、Service、Mapper 三层架构。Controller 层定义接口的请求方式、请求路径以及请求参数等，负责接收请求，调用 Service 处理业务逻辑；Service 层处理核心业务逻辑和事务，以及调用 Mapper 或外部服务；Mapper 层与数据库进行交互，对于一些复杂的多表操作，由于 Mybatis-plus 只是提供了一些简单的单表操作，因此我们需要在 xml 文件中编写相应的 SQL 语句。对于数据查询操作，封装了查询请求基类 QueryRequest，便于其他模块复用统一的分页（页号，页大小）与排序规则（排序字段、升序开关）属性，各个模块可以根据需求继承该类来扩展特定的业务查询维度。控制层在收到业务层的返回结果后，统一由 FebsResponse 包装后响应给前端。

5.1.1 部门投入 BU 工时分布

部门当月投入到各 BU（业务单元）的工时分布，是企业管理人力资源调配的重要可视化视图。当前界面默认加载当月的公司整体 BU 工时饼图，用户也可通过年份加月份进行历史数据查询。该模块还可展示某 BU 的工时详情，包括总工时、平均工时、工时占比、加班工时及工时趋势等关键指标。这些指标数据为项目成本核算与业务决策提供客观的数据支撑。

（1）类图设计

WorkHourController 作为工时管理模块的控制层类，主要负责接收前端发送的工时统计请求，并将请求参数传递给业务层进行处理。WorkHourService 是工时管理模块的业务接口，用于定义工时统计相关的业务方法，保证控制层与具体业务实现之间保持松耦合。WorkHourServiceImpl 是该接口的实现类，负责完成工时数据查询、组织信息处理、项目数据关联以及缓存读写等核心业务逻辑。WorkHourQueryDTO 用于封装前端传入的查询条件，包括年份、月份、组织节点和 BU 名称等参数，从而避免接口参数过于分散。FebsResponse 用于统一封装接口响应结果，保证前后端数据交互格式的一致性。Mapper 相关接口则主要负责数据库访问，其中 WorkHourMapper 用于查询工时数据，OrgMapper 用于获取组织结构信息，TbEmpBaseinfoMapper 用于查询员工信息，ProjectMapper 用于获取项目相关数据。

由图 5-1 可知，WorkHourController 与 WorkHourService 之间存在依赖关系，控制层并不直接处理具体业务逻辑，而是通过调用业务接口完成工时统计请求的转发与结果返回。

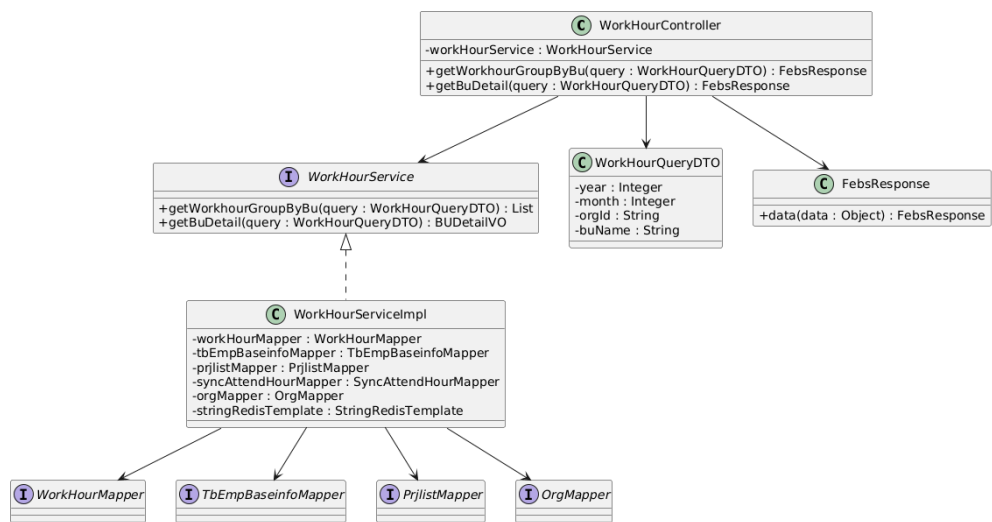


图 5-1 BU 工时统计类图

WorkHourServiceImpl 实现了 WorkHourService 接口，具体负责 getWorkhourGroupByBu 和 getBuDetail 等业务方法的实现，这种面向接口的设计方式有利于降低模块之间的耦合度，也便于后续对业务逻辑进行扩展和维护。

(2) BU 工时分布统计

BU 工时分布统计用于展示指定年月、指定组织范围内各 BU 的工时占比情况，其核心业务流转如图5-2所示。

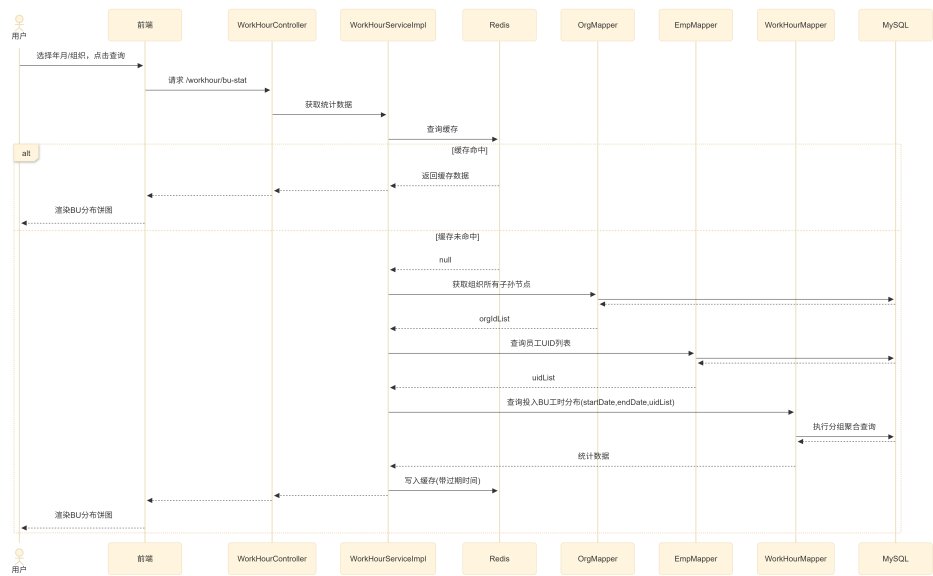


图 5-2 BU 工时饼图时序图

用户在前端页面中选择查询年月和组织节点后，点击查询按钮发起统计请求。前端将查询条件提交至后端请求处理层，后端首先对年月、组织节点等参数进行合法性校验。校验通过后，业务层根据组织节点获取其下属组织范围，之后查询这些组织下的员工列表。随后，系统结合查询时间参数和员工 UID 列表，从工时数据表中按 BU 维度进行分组聚合，计算各 BU 的工时分布结果。最终，后端将统计结果封装后返回前端，前端通过 ECharts 饼

图组件完成可视化展示。

（3）BU 工时详情查询

当用户需要进一步查看某个 BU 的工时详情，可以点击饼图中对应位置，即可弹框展示详情，其业务逻辑如图5-3所示。

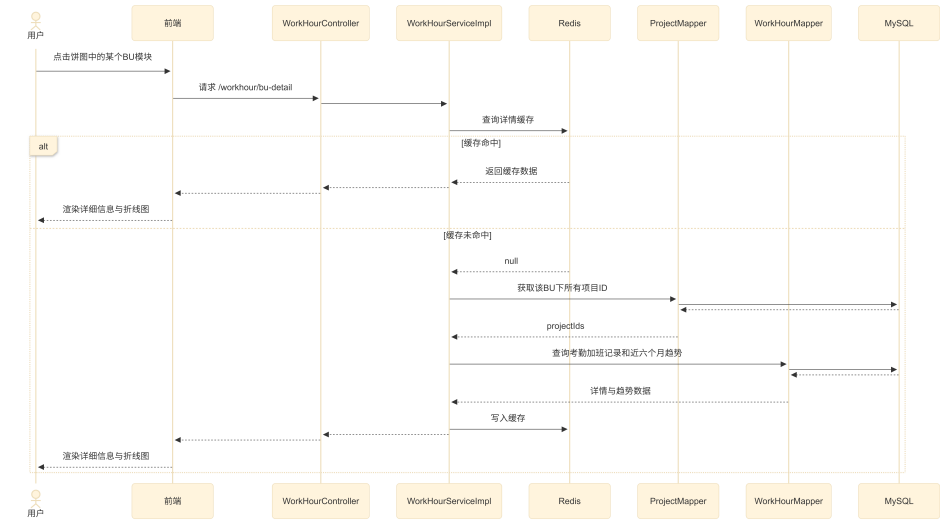


图 5-3 BU 详情时序图

前端根据用户点击的 BU 信息发起详情查询请求。后端接收到请求后，对 BU 标识和查询时间等参数进行校验。校验通过后，业务层首先去项目表里查询出归属与这个 BU 的项目 ID 列表，并基于项目和时间范围进一步查询加班工时、以及近六个月的工时变化趋势。查询完成后，系统将基础信息、工时汇总结果和趋势数据统一封装返回前端，由前端弹窗展示基础信息面板和工时趋势折线图。

（4）查询性能优化

在 BU 工时统计模块的初始实现中，接口响应时间较长。通过分析 SQL 执行计划发现，工时数据查询未能有效利用已建立的日期索引，而是出现了全表扫描。进一步分析后发现，数据库工时表中的日期字段精确到天，而初始实现中直接使用 DATE_FORMAT 等日期格式化函数进行年月匹配，导致查询条件作用在函数计算结果上，使数据库无法正常利用日期索引^[27]。

针对上述问题，系统在业务层对年月参数进行预处理，将其转换为该月份的起止时间范围。数据库查询时不再使用日期格式化函数，而是通过时间范围条件过滤工时记录，使 SQL 能够命中日期索引，从而减少扫描数据量并降低接口响应时间。

此外，考虑到该模块数据具有“读多写少”的特点，系统引入 Redis 缓存机制保存统计结果。后续请求若命中缓存，则直接返回缓存数据；若缓存未命中，系统再访问数据库完成统计计算，并将结果写入 Redis。缓存过期时间根据数据变化频率进行差异化设置：当月数据仍可能持续变化，缓存时间设置为一小时；历史月份数据变化频率较低，缓存时间

设置为二十四小时。同时，系统在缓存过期时间中加入随机抖动，避免大量缓存键在同一时间集中失效，从而降低缓存雪崩风险。

5.1.2 项目基本信息管理

项目基本信息管理模块主要围绕项目基础数据的查询、维护、导出和交付历史记录查询展开，其核心类包括控制层类、业务接口类、业务实现类、数据访问类、查询参数类。

其中，`TbProjectBaseinfoController` 是项目基本信息模块的控制层类，主要负责接收前端请求，并根据不同接口调用对应的业务方法。该类中包含分页查询、新增项目、修改项目、导出项目信息以及查询历史记录等方法，是前端访问该模块功能的入口。`ITbProjectBaseinfoService` 是业务层接口，用于定义项目基本信息相关的业务操作，包括项目信息分页查询、项目创建、项目更新和历史记录查询等。`TbProjectBaseinfoServiceImpl` 是业务接口的实现类，负责完成具体的业务处理逻辑，并通过数据访问层获取或保存相关数据。

在数据访问层中，`TbProjectBaseinfoMapper` 主要用于操作项目基本信息表，完成项目信息的查询、新增和修改等数据库操作；`TbProjectBaseinfoHistoryMapper` 主要用于操作项目历史记录相关数据，为项目变更记录查询提供数据支持。`ProjectQueryRequest` 是项目查询请求对象，用于封装前端传入的查询条件，如 BU 名称和项目名称等。由于项目列表查询通常需要分页和排序功能，因此 `ProjectQueryRequest` 继承了 `QueryRequest`。

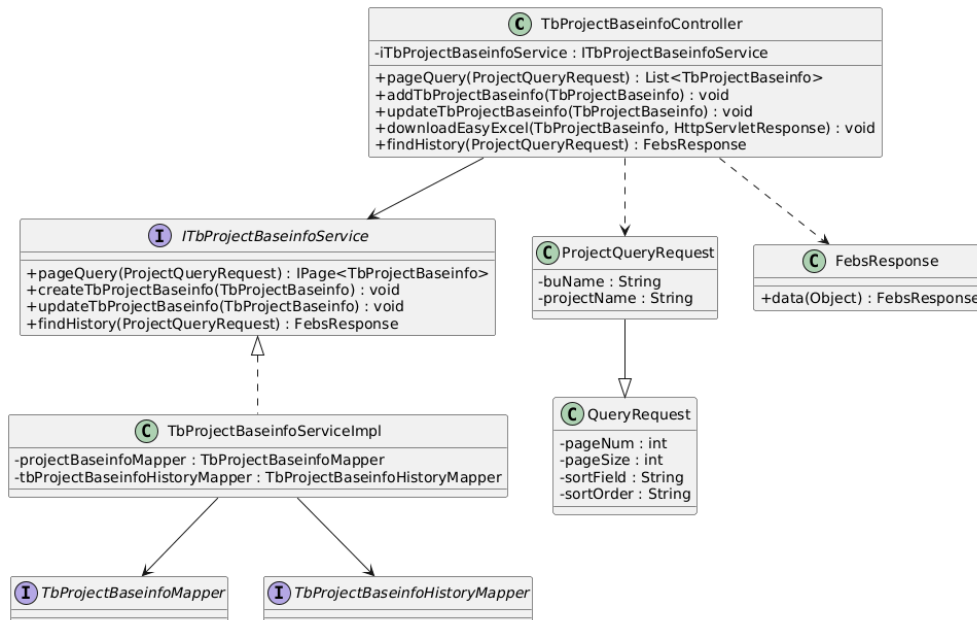


图 5-4 项目基本信息管理类图

由图 5-4 可知，`TbProjectBaseinfoController` 依赖 `ITbProjectBaseinfoService` 完成业务调用，避免控制层直接处理数据库操作。`ITbProjectBaseinfoService` 由 `TbProjectBaseinfoServiceImpl` 实现，具体的项目信息处理逻辑集中在实现类中完成。`TbProjectBaseinfoServiceImpl` 进一步调用 `TbProjectBaseinfoMapper` 和 `TbProjectBaseinfoHistoryMapper` 获取项目基础数据

及历史记录数据。由于项目列表查询通常需要分页和排序功能，因此 `ProjectQueryRequest` 继承 `QueryRequest`。

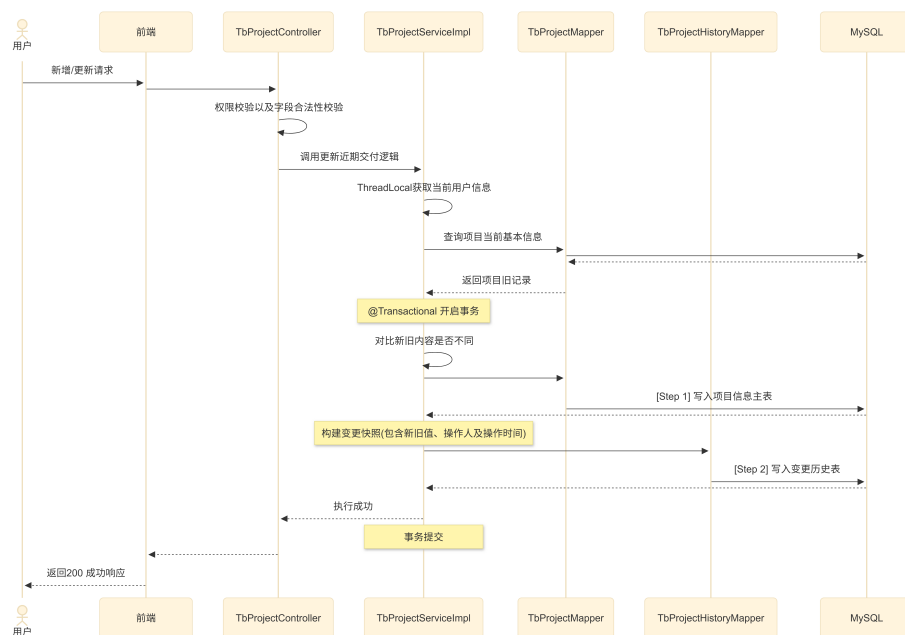


图 5-5 项目信息管理时序图

针对项目基本信息的“新增与更新”流程，系统在后端实现上重点保障了数据的安全性与可追溯性，其核心业务流转如图 5-5 所示。我们在 `TbProjectBaseinfoController` 的新增和更新方法上加上了 `@RequiresPermissions` 注解，前端发起的 HTTP 请求会先经过请求处理层的 JWT 校验，通过后 Shiro 框架会拦截加了注解的方法，在方法执行前先查询预热到缓存的权限数据，如果没有就去数据库查询，检查当前用户是否有对应权限，并利用 `@Valid` 完成参数的合法性校验。进入业务层后，系统采用“主表 + 历史表”的双写策略：首先将最新的项目信息更新至 MySQL 主表；随后，通过构建当前操作的完整数据快照（涵盖新旧值、操作人及操作类型），并写入项目信息快照表。这种快照机制便于后续的项目审计与数据回滚操作。由于部分字段比如“近期交付阀点”和“近期阀点日期”修改比较频繁，可能导致快照表的体积变得太大，因此针对这部分的修改历史保存到交付历史变更表中。由于更新操作修改了多张表，需要在业务层的方法上加上 `@Transactional` 注解，将多表更新操作限制在单一事务中，以确保原子性和一致性。需要注意的是，在实际开发中，并不是在方法上加注解就可以高枕无忧了，事务包裹的范围应该尽可能小，否则可能会导致长事务的出现，很容易耗尽连接池导致数据库崩溃，同时长期持有的锁会引发大范围阻塞，并因阻碍日志清理而拖垮数据库性能。此外还要考虑注解失效的情景，避免在开发中犯这类错误。

5.1.3 缺陷管理

在企业实际研发过程中，项目缺陷数据来源较为分散，企业内部同时使用了 Trinity、Jira 以及 BugClose 这三个缺陷管理平台。其中，Trinity 为企业内部平台，另外两个是主流的缺陷管理工具。三个平台在使用过程中都会产生缺陷相关数据，如缺陷编号、缺陷标题、所属项目、缺陷状态、严重程度、责任人、创建时间、关闭时间等，这些数据共同反映了缺陷从发现到解决的处理过程。但是各平台的字段设计也存在一些差异，在系统中会用三张表格进行存储，以保留各平台原有字段及业务含义。缺陷管理模块需要对这些缺陷表进行统一管理，聚合统计出缺陷的解决比率。通过数据汇聚与指标计算，为项目质量人员提供多维度的缺陷质量看板。

BugController 作为控制层，负责接收前端请求，并调用对应的业务接口完成缺陷相关操作。BugService 用于定义缺陷管理的主要业务方法，BugServiceImpl 负责具体业务逻辑的实现。ProjectClusterService 主要用于提供项目聚类相关的数据支持，在缺陷统计和聚类展示中起到辅助作用。数据访问层中，BugMapper 负责缺陷数据的查询与维护，OrgMapper 用于获取组织相关信息。BugQueryRequest 用于封装缺陷查询条件，并继承 QueryRequest 中的通用查询参数；FebsResponse 用于统一封装接口返回结果。

缺陷管理类图如图5-6所示。

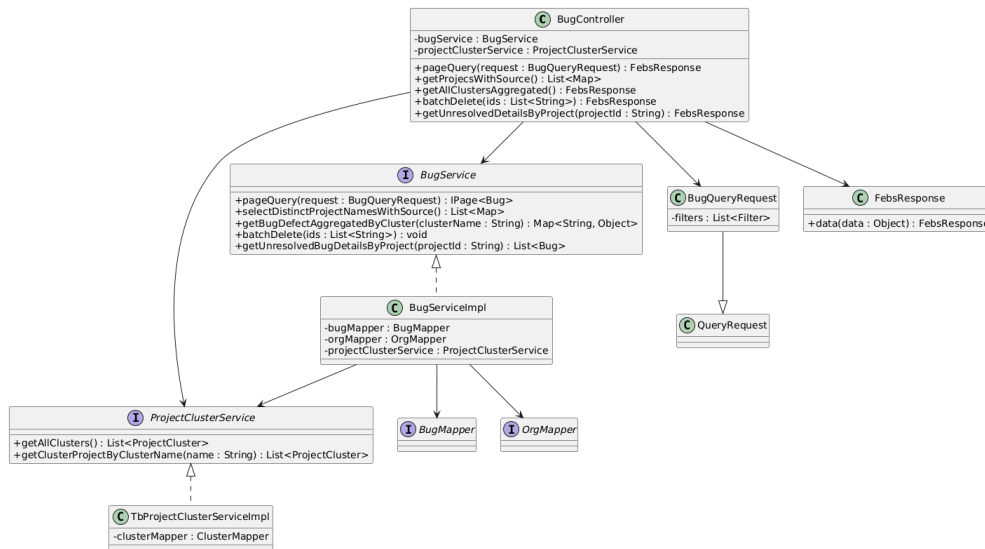


图 5-6 缺陷管理类图

(1) 项目聚类缺陷统计

缺陷管理部分主要包括缺陷数据的分页查询以及项目聚类缺陷统计，其中，项目聚类缺陷统计的业务流程如图5-7所示。项目聚类缺陷统计主要用于对多个项目组成的聚类进行缺陷数据汇总分析。系统首先从项目聚类关系表中获取全部聚类名称，并根据聚类名称查询其包含的项目列表。随后，系统以项目为基本统计单元，分别从 Trinity 缺陷表、Jira 缺陷表和 BugClose 缺陷表中提取缺陷数据，并计算当前未解决缺陷数、新增缺陷数、解决

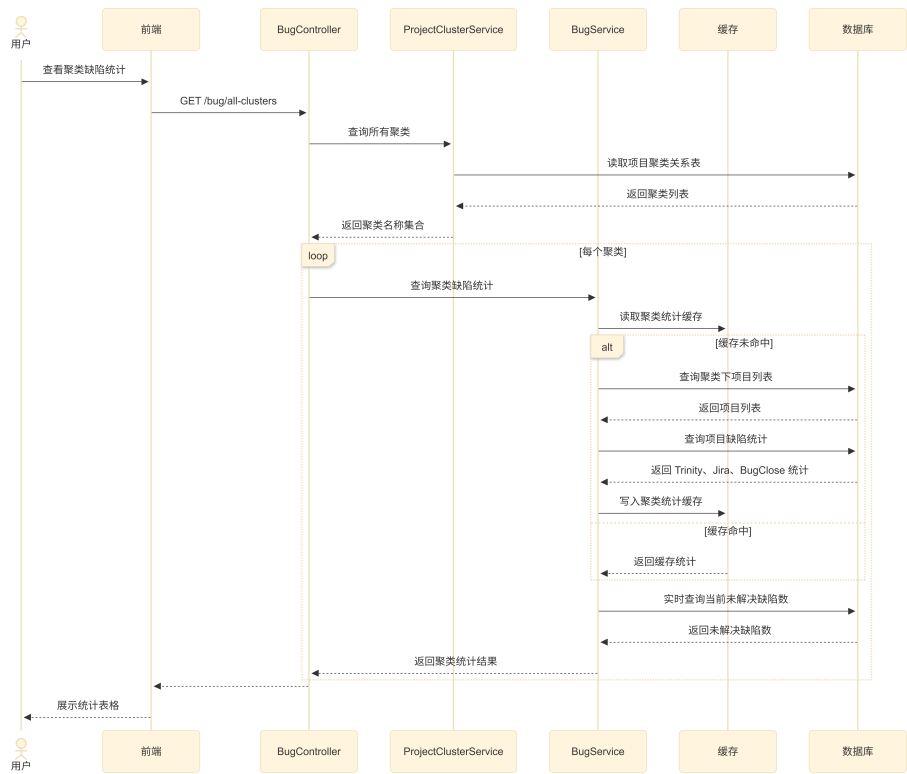


图 5-7 项目聚类缺陷统计时序图

缺陷数、超期未解决缺陷数以及关闭率等指标。最后以表格的形式展示给用户，对于一些关闭率低的项目聚类，还设置了定时任务发送飞书预警消息来提醒。

考虑到聚类缺陷统计涉及多个数据表，查询成本较高，系统在设计中引入缓存机制。对于昨日新增、昨日解决、总缺陷数、关闭数以及聚类级历史统计等变化频率较低的数据，系统使用缓存保存统计结果；对于当前未解决缺陷数等实时性要求较高的数据，则在接口调用时进行实时计算，并与缓存结果合并后返回。该设计在保证统计结果实时性的同时，降低了数据库访问压力，提高了聚类缺陷看板的响应效率。

（2）缺陷查询深分页优化

系统前端提供了对三类缺陷数据表的分页查询功能。由于缺陷表数据量较大，当用户查询较靠后的页码时，普通分页方式的查询效率会明显下降。其原因在于，对于 `LIMIT offset, N` 语句，数据库需要先扫描 `offset + N` 条记录，再丢弃前 `offset` 条数据，最终返回当前页所需的 `N` 条记录。当偏移量较大时，会产生大量无效扫描；如果查询字段较多，还可能带来较多回表操作，从而影响接口响应速度^[28]。

针对上述问题，系统在深分页查询中采用延迟关联的优化思路。该方式先在子查询中根据筛选条件和排序字段查询当前页对应的主键，再通过主键关联原表获取完整缺陷信息。相比直接查询全部字段，延迟关联可以减少大量无效数据的回表操作，从而提升大数据量分页查询场景下的查询效率。普通分页查询与延迟关联分页查询的 SQL 对比如图 5-8 所示。

普通分页

```

1 SELECT * FROM tb_sync_bug
2 WHERE severity = 'A'
3 ORDER BY created_date DESC
4 LIMIT 100000, 20;

```

延迟关联分页

```

1 SELECT b.* FROM tb_sync_bug b
2 INNER JOIN (
3     SELECT id, created_date FROM tb_sync_bug
4     WHERE severity = 'A'
5     ORDER BY created_date DESC, id ASC
6     LIMIT 100000, 20
7 ) page_ids ON page_ids.id = b.id
8 ORDER BY page_ids.created_date DESC, page_ids.
   id ASC;

```

图 5-8 深分页优化前后 SQL 语句横向对比

5.1.4 重大问题管理

重大问题管理模块面向项目执行过程中影响范围较大、处理周期较长或需要跨部门协调的问题，大致可分为问题登记、问题处理和问题关闭三个阶段。具体流程如图5-9所示。

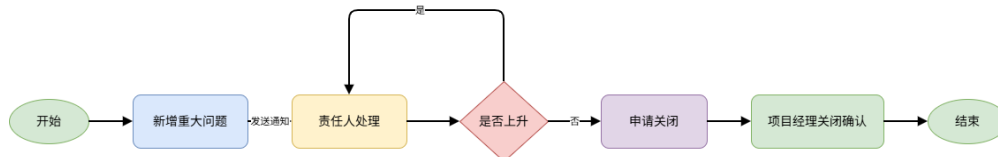


图 5-9 重大问题流程图

重大问题管理功能围绕 TbProjectSeriousProblem 业务对象展开，主要实现重大问题的分页查询、新增修改、数据导出以及飞书提醒等操作。控制层 TbProjectSeriousProblemController 负责接收前端请求，并将具体业务处理交给服务层完成；服务层 TbProjectSeriousProblemServiceImpl 负责重大问题数据的查询处理、人员信息补全以及提醒消息组织；数据访问层 TbProjectSeriousProblemMapper 负责完成重大问题数据的数据库查询。除此之外，TbProjectSeriousProblemQueryRequest 用于封装前端传入的查询条件，使接口参数更加清晰。

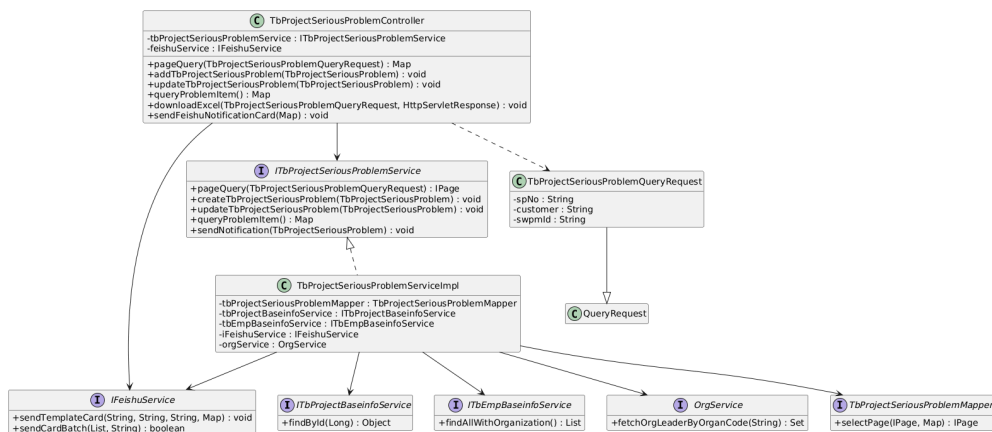


图 5-10 重大问题管理类图

重大问题管理类图如图 5-10所示。从类之间的关系来看，控制层依赖 ITbProjectSeriousProblemService 完成核心业务操作。TbProjectSeriousProblemServiceImpl 作为业务接口

的实现类，通过 TbProjectSeriousProblemMapper 查询重大问题数据，调用项目、员工和组织相关服务接口查询飞书消息接收人列表，然后调用飞书服务的 sendTemplateCard() 方法完成消息卡片的发送。

重大问题管理的核心业务流程如图5-11所示。用户进入列表页面后，系统首先根据当前登录用户的身份判断其权限，查询并展示其可见的问题记录。用户可通过顶部筛选栏按多维度条件进行检索，也可点击”新增”按钮创建新的重大问题。

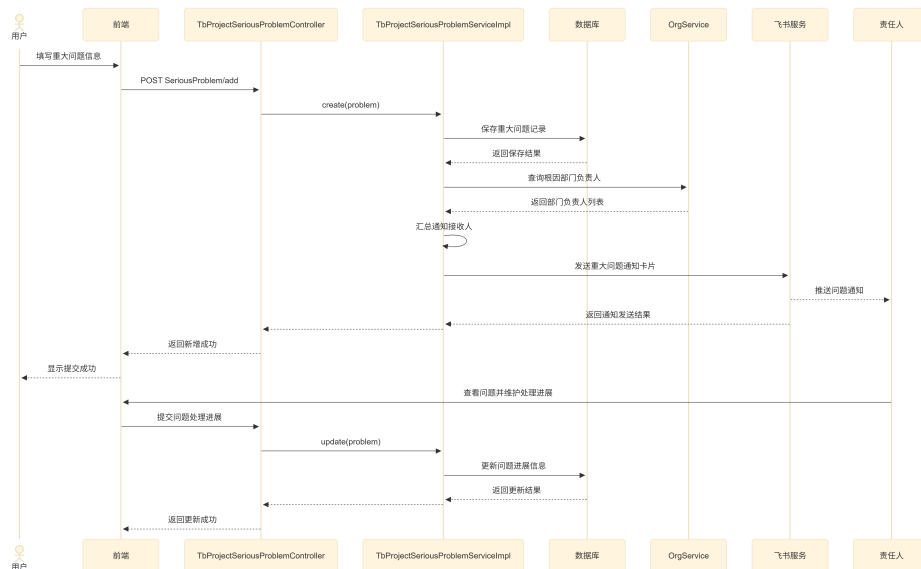


图 5-11 重大问题时序图

在问题登记阶段，用户填写重大问题的基础信息和首问责任人。系统保存记录后，根据问题涉及的责任人生成通知内容，并通过飞书消息推送给相关人员，使问题能够及时进入处理环节。在问题处理阶段，由责任人来填写问题的技术原因和解决方案；当问题需要更高层级协调时，可将问题标记为“需要上升”，并指定上升责任人，由其介入协调资源后继续推进问题处理。当问题需要关闭时，用户需要补充实际解决日期、最终责任人、责任分类、技术原因、技术解决方案等信息。系统根据计划解决日期和实际解决日期自动判断问题状态：实际解决日期不晚于计划解决日期时，问题状态为按时关闭；实际解决日期晚于计划解决日期时，问题状态为超期关闭；若超过计划解决日期仍未填写实际解决日期，则问题保持超期未关闭状态。通过该设计，模块能够对重大问题从发现、处理、上升到关闭的全过程进行记录和约束。

5.2 数据库设计

为说明系统中各业务数据对象之间的关系，本文首先给出数据库的总体逻辑 ER 图，如图 5-12 所示。该图从业务对象角度抽象出组织、员工、项目、缺陷、重大问题、项目聚类和聚类缺陷快照等核心实体，并描述各实体之间的主要关联关系。后续小节将在此基础上分别说明关键数据表的字段设计。

5.2.1 概念逻辑设计

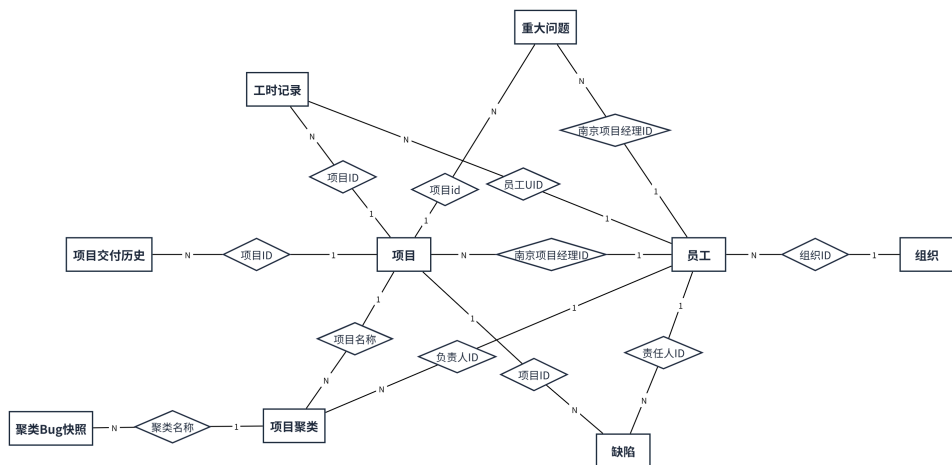


图 5-12 数据库 E-R 图

系统数据库以项目为主要业务对象进行组织。项目在系统中既是缺陷统计、工时记录和重大问题管理的归属对象，也是项目聚类分析的基础单元。因此，在总体 ER 图中，项目实体位于业务数据关系的中心位置，并分别与缺陷、重大问题、工时记录、项目交付历史和项目聚类等实体建立联系。

组织和员工实体用于描述系统中的人员基础数据。组织实体记录组织编号、组织编码以及上级组织编号，用于表示企业内部的层级结构；员工实体记录员工编号、员工 UID 以及所属组织。员工与组织之间是一对多关系，即一个组织下可以包含多名员工，而每名员工归属于一个组织。

项目实体记录项目状态、项目所属 BU、南京项目经理、当前里程碑、近期交付阀点及阀点日期等信息。项目与员工之间通过项目经理的 id 建立关联，一个项目经理可以负责多个项目。项目交付历史实体用于记录项目近期交付的变更情况。该实体与项目实体之间是一对多关系，表示同一项目在执行过程中可能产生多次交付节点调整记录。

缺陷实体用于保存项目缺陷的主要信息。缺陷与项目之间是一对多关系，一个项目可以对应多条缺陷记录；缺陷与员工之间通过责任人建立关联，用于定位缺陷处理人员。

重大问题实体用于记录项目执行过程中需要重点跟踪的问题。重大问题与项目之间是一对多关系，即一个项目可以产生多条重大问题记录。

项目聚类是指将多个相关项目归为一组，以组为单位进行缺陷统计。系统通过聚类名称将若干项目归为同一统计对象，并为聚类配置负责人。项目聚类与项目之间形成一对多关系，一个聚类可以包含多个项目；项目聚类与员工之间通过负责人建立关联，用于表示该聚类统计对象的管理责任人。聚类 Bug 快照与项目聚类之间是一对多关系，同一聚类可以按日期保存多条快照记录，从而支持缺陷趋势分析。

总体来看，该数据库模型以项目为中心，将人员组织、缺陷数据、重大问题、工时记录、交付历史和聚类统计数据连接起来。组织和员工实体提供人员归属关系，项目实体承载项目基础信息，缺陷和重大问题实体记录项目执行过程中的质量问题，项目聚类和聚类Bug快照实体则面向跨项目统计分析。由于部分业务数据来源于外部同步表，实际物理表中并非所有关系都采用数据库外键约束，图中的关系主要用于描述系统业务对象之间的逻辑关联。

5.2.2 数据表设计

在数据库总体设计的基础上，本节进一步说明系统主要业务表的结构设计。

- (1) BU 工时统计主要用于分析各个组织部门在业务单元的工时投入情况，涉及的实体包括组织树（tb_org_tree）、员工基础信息（tb_emp_baseinfo）、项目列表（tb_project_baseinfo_new）和工时记录（tb_sync_wh），其关系如图5-13所示。

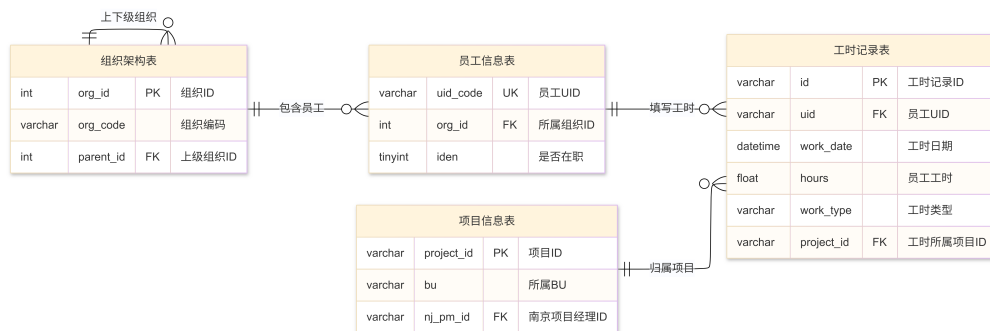


图 5-13 工时数据库关系图

tb_org_tree 用于保存组织树结构，通过 parent_id 与 org_id 的自关联关系表示企业内部组织层级。tb_emp_baseinfo 用于保存员工基础信息，其中 org_id 表示员工所属组织，uid_code 是员工在系统中的唯一标识。工时记录表 tb_sync_wh 存储从 Trinity 系统同步而来的员工填报工时数据，通过 uid 与员工表中的 uid_code 建立逻辑关联，通过 project_id 与项目基础信息表中的项目建立逻辑关联。

- (2) 项目基础信息主要用于保存项目的主数据及其变更记录，为项目查询、工时统计、缺陷分析和重大问题管理提供统一的项目维度。该部分涉及的实体包括员工基础信息（tb_emp_baseinfo）、项目基础信息（tb_project_baseinfo_new）、项目基础信息历史记录（tb_project_baseinfo_new_his）和项目交付阀点历史记录（tb_project_delivery_history），其关系如图5-14所示。

tb_project_baseinfo_new 用于保存项目当前基础信息，包括项目编号、项目状态、所属 BU、客户名称、南京项目经理、当前里程碑、近期交付阀点和阀点日期等字段。其中，project_id 是项目的唯一标识，nj_pm_id 用于关联员工基础信息表中的项目经理。项目基础信息历史表 tb_project_baseinfo_new_his 用于保存项目基础信息



图 5-14 项目基础信息数据库关系图

的变更记录，记录项目状态、所属 BU、里程碑计划时间、提交人和提交时间等信息。项目交付节点历史表 `tb_project_delivery_history` 用于保存近期交付节点的调整记录，通过 `project_id` 与项目基础信息表建立逻辑关联，从而保留项目交付计划的变更过程。

(3) 项目聚类缺陷统计主要用于对多个项目的缺陷数据进行汇总分析，支撑聚类维度的质量看板展示。

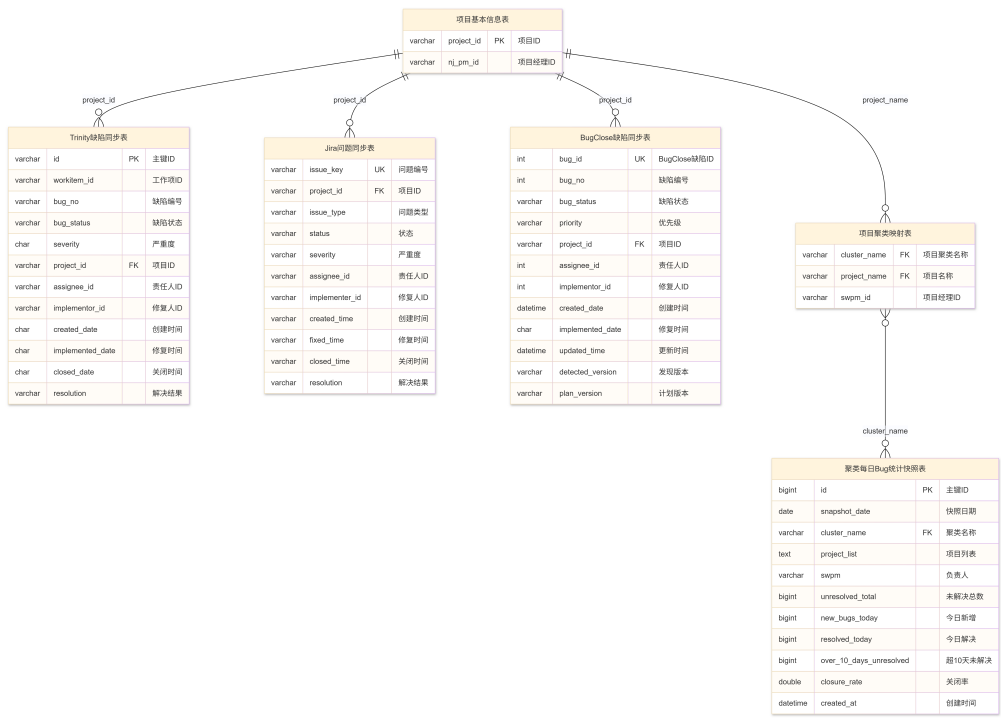


图 5-15 缺陷数据库关系图

该部分涉及的实体包括项目基础信息（`tb_project_baseinfo_new`）、Trinity 缺陷同步表（`tb_sync_bug`）、Jira 问题同步表（`tb_sync_jira`）、BugClose 缺陷同步表（`tb_sync_bugclose`）、项目聚类映射表（`tb_project_cluster`）和聚类缺陷统计快照表（`tb_bug_`

cluster_daily_snapshot), 其关系如图5-15所示。

tb_sync_bug、tb_sync_jira 和 tb_sync_bugclose 分别保存来自不同缺陷系统的同步数据, 主要字段包括缺陷编号、项目编号、缺陷状态、严重度、责任人、修复人、创建时间、修复时间和关闭时间等。由于三类缺陷数据来源不同, 字段命名和状态取值存在差异, 系统在统计时根据项目编号或项目名称进行匹配, 并在业务层统一计算当前未解决数、新增数、解决数和关闭率等指标。项目聚类映射表 tb_project_cluster 用于维护聚类名称与项目名称之间的对应关系, 系统根据该表确定一个聚类包含的项目范围。聚类缺陷统计快照表 tb_bug_cluster_daily_snapshot 用于保存聚类维度下的每日统计结果, 包括快照日期、聚类名称、项目列表、当前未解决总数、今日新增、今日解决、超期未解决数量和关闭率等字段, 用于后续趋势分析和历史数据查询。

- (4) 重大问题管理主要用于记录项目执行过程中需要重点跟踪的问题, 并支撑问题登记、责任人处理、问题上升和闭环管理。该部分涉及的实体包括项目基础信息 (tb_project_baseinfo_new)、员工基础信息 (tb_emp_baseinfo)、组织架构 (tb_org_tree) 和项目重大问题表 (tb_project_serious_problem_new), 其关系如图5-16所示。

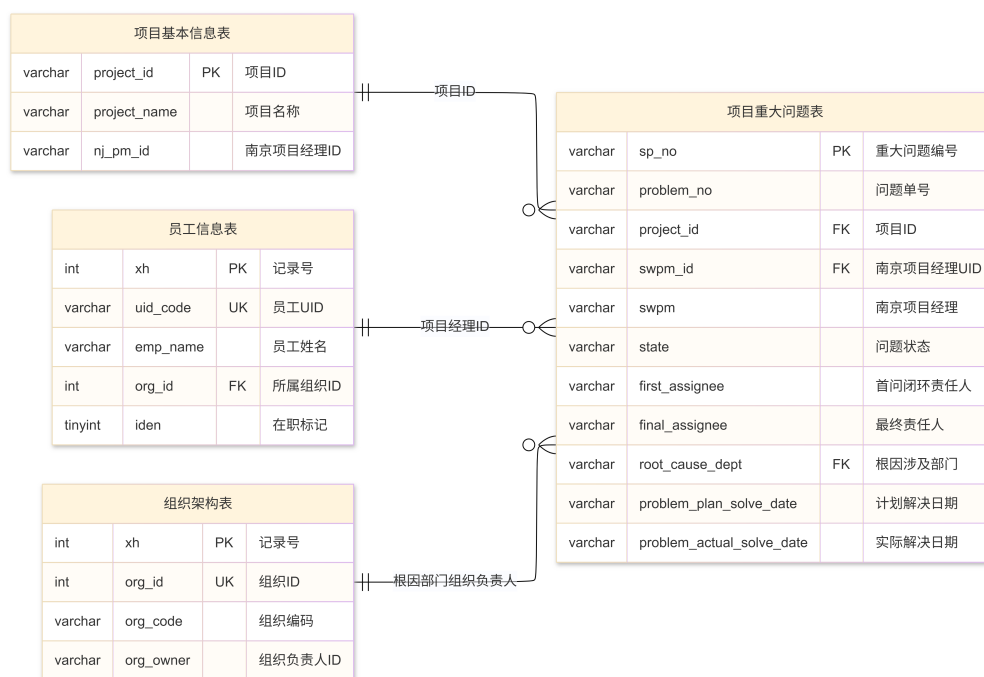


图 5-16 重大问题管理数据库关系图

tb_project_serious_problem_new 是重大问题管理模块的核心表, 以 sp_no 作为重大问题记录编号, 保存问题单号、项目编号、南京项目经理、问题状态、首问闭环责任人、最终责任人、根因涉及部门、计划解决日期和实际解决日期等信息。项目基础信息表通过项目编号或项目名称与重大问题记录建立逻辑关联, 用于确定问题所属项目及其项目经理信息。员工基础信息表通过 uid_code 与重大问题表中的项目经理、责

任人等字段建立业务关联，用于消息通知和责任人识别。组织架构表通过组织编码和组织负责人信息与根因部门字段关联，系统可根据根因涉及部门查询相应负责人，并将其纳入重大问题通知范围。

5.3 本章小结

本章重点论述了系统核心业务模块的详细设计，介绍了各个功能模块有关的类和接口以及关联关系，同时详细展示了几个关键点的具体设计和优化，给出了数据库中相关表结构的设计。下一章节，将基于系统的详细设计方案，展开对系统实现与测试的介绍。

第六章 系统测试

系统测试是保证系统质量的关键，核心目的是验证系统是否满足需求规格，并确保其能够稳定、安全地交付给用户使用。本章对所实现的功能进行测试。对系统的项目管理、缺陷管理、重大问题管理、BU 工时统计这几个核心功能模块进行展示与测试，最后从系统的响应时间指标等对系统的非功能性需求进行测试和评估。

6.1 测试环境

为保证系统测试结果的真实性与有效性，首先需要明确软件的开发环境。以下列出了服务端、数据库、缓存中间件及前端界面的运行基础配置参数与版本信息，为后续各项测试的顺利开展提供一致的基准支撑。

表 6.1 软件环境

配置项	版本/参数
JDK	Java 8 (1.8)
Spring Boot	2.1.0.RELEASE
MyBatis-Plus	3.4.0
MySQL	8.0.29
Redis	5.0.14.1
前端框架	Vue 2
构建工具	Maven 3.9.6
Nodejs	16

6.2 功能测试

功能性测试旨在验证系统各项业务模块是否严格符合业务规格说明书的需求。本节将从实际的企业运营场景出发，依序对 BU 工时统计分析、项目基本信息管理、缺陷管理以及重大问题管理四个核心业务模块展开黑盒测试，以确保核心功能的逻辑正确性与数据完整性。

6.2.1 部门投入 BU 工时分布功能测试

BU 工时统计分析主要为了让管理层能在宏观上把控整个南京在各个 BU 的工时投入占比，以及特定 BU 模块的详细工时，便于即时对比 BU 的整体工时投入情况和预算数据，从而进行及时的管理干预。用户进入该界面后，系统默认展示当月工时数据及投入 BU 的占比情况，页面展示效果如图6-1所示，用户可以点击部门下拉框，就可以看到树状的组织结构进行选择，也可以点击时间进行选择。点击某个 BU 模块后就会弹出 BU 详情对话框，显示总工时、平均工时、加班工时等数据，并以折线图的形式展示近六个月以来公司投入

到该 BU 的工时趋势，页面效果如图6-2所示。

表 6.2 BU 工时统计接口测试用例

编号	测试用例	测试步骤	预期结果	测试结果
TC01	BU 工时分布查询	用户点击组织下拉框选择一个组织节点,点击时间日历框选择年月,点击搜索按钮	返回各 BU 的工时数、占比，渲染成饼图	符合预期
TC02	BU 详情查询	点击饼图的某一块，选定要查询的 BU 名称	返回该 BU 的总工时、平均工时、加班工时、趋势	符合预期

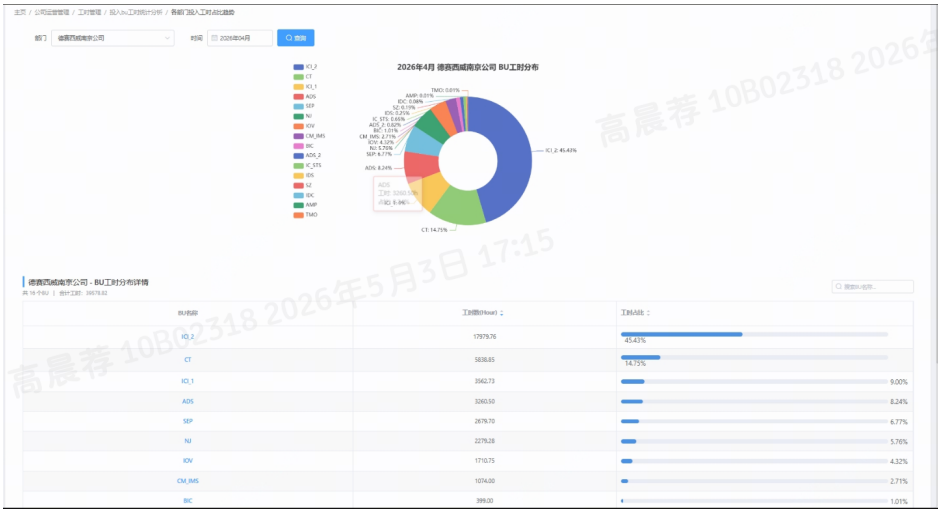


图 6-1 BU 工时分布饼图

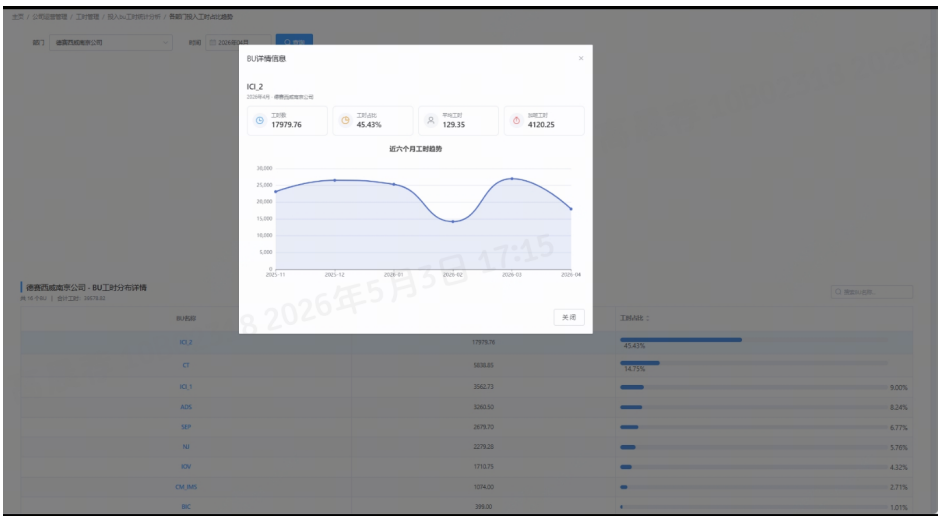


图 6-2 BU-ICT2 详情

6.2.2 项目基本信息管理测试

项目基本信息管理模块提供项目全生命周期的基础数据维护功能，即项目管理的立项阶段。本节对该模块的核心功能进行测试验证。图6-3展示的是项目管理界面，包括项目信息的增删改查、Excel 导出等，用户更新项目信息后，可以查看更改历史，如图6-4所示。

表 6.3 项目基础信息管理测试用例

编号	测试用例	测试步骤	预期结果	测试结果
TC03	按条件组合筛选	在表格上方选择筛选条件，然后点击查询	返回符合筛选条件的项目	符合预期
TC04	新增项目	填写项目表单，然后点击提交	保存成功，列表中出现新项目	符合预期
TC05	修改项目信息	点击要修改项目的操作列按钮，修改数据后点击保存	更新成功	符合预期
TC06	导出 Excel	点击导出按钮	下载包含当前数据的xlsx 文件到本地	符合预期
TC07	获取项目详情	点击高亮的项目名称	进入项目的详细信息界面	符合预期
TC08	查看项目交付历史	在项目周报详情界面，点击近期交付阀点旁边的查看历史	在界面右侧弹出一个抽屉，以时间轴的形式展示修改历史	符合预期

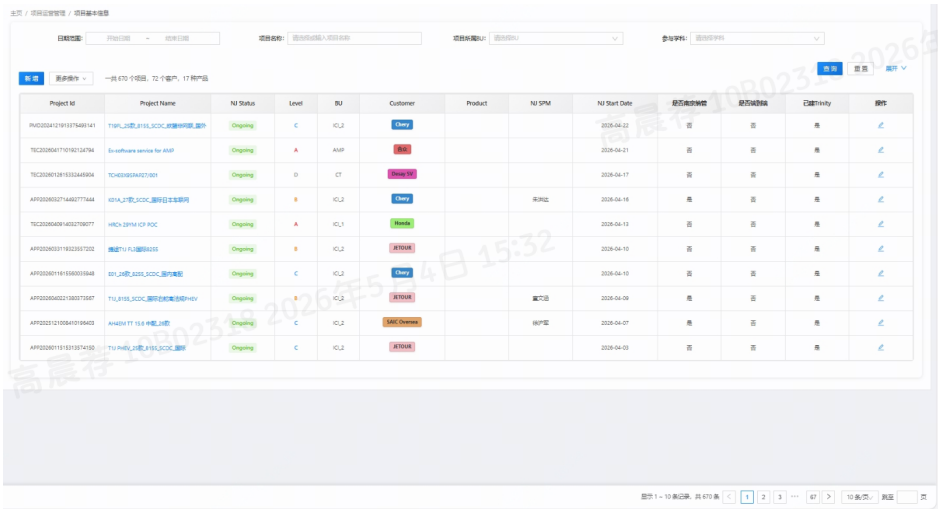


图 6-3 项目基本信息管理

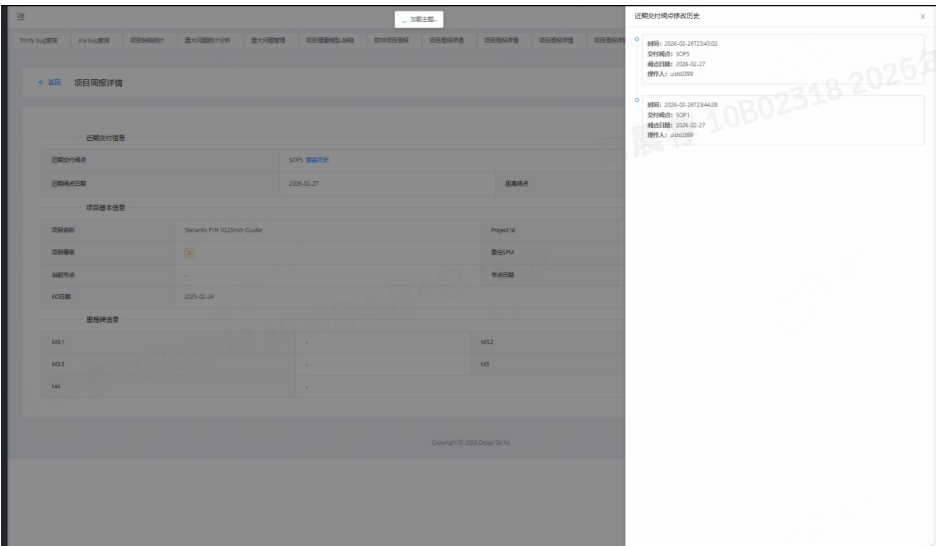


图 6-4 项目交付历史

6.2.3 缺陷管理测试

缺陷管理模块整合了 Trinity、Jira 和 BugClose 三个数据源的缺陷数据，查询结果如图6-5所示。

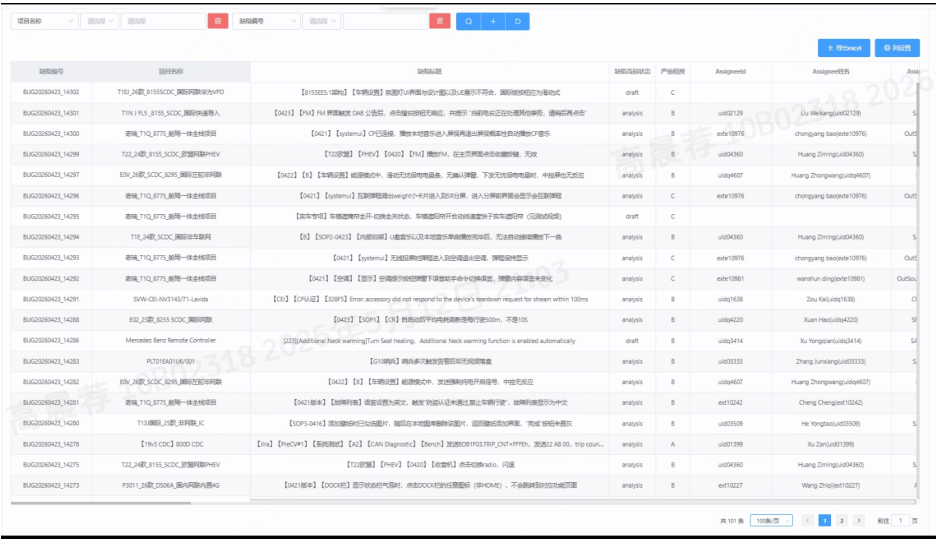
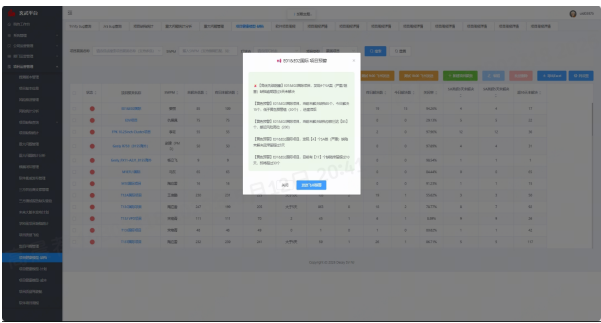
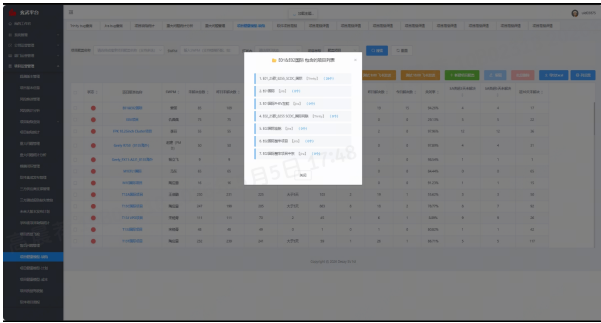


图 6-5 项目缺陷报表

除了提供基础的查询功能，还包括以聚类为维度提供多项目缺陷看板。用户点击菜单侧边栏后，主页面以表格形式展示各聚类的未解决总数、新增数、解决数、关闭率等关键指标，并以红黄绿灯状态标识项目健康度，点击状态灯会弹出预警弹窗，点击按钮可以提醒相关人员，如图6-6(a)所示。点击聚类名称可进一步查看其下各项目的缺陷详单，如图6-6(b)所示。



((a)) 缺陷飞书预警



((b)) 项目聚类缺陷详单

表 6.4 缺陷管理接口测试用例

编号	测试用例	测试步骤	预期结果	测试结果
TC09	缺陷查询	点击缺陷查询菜单 栏下的分类	各自返回对应数据源 的缺陷列表	符合预期
TC10	聚类统计列表 查询	在该页面按聚类名 称/SPM/灯状态筛选	表格按条件正确显示	符合预期
TC11	发送飞书预警	点击状态灯，弹出预 警话术，	弹出预警话术，可发送 飞书预警	符合预期
TC12	聚类项目列表 与缺陷详单	点击聚类名称	弹窗显示聚类下项目 列表和项目的缺陷详 单	符合预期
TC13	聚类管理	新建/编辑/批量删除 聚类	操作成功，前端弹出提 示消息，列表实时刷新	符合预期

6.2.4 重大问题管理测试

重大问题管理模块用于记录和跟踪项目执行过程中出现的重大异常事件，覆盖问题从发现登记、根因分析、方案制定到实施关闭的完整闭环流程，重大问题管理页面如图6-7所示。

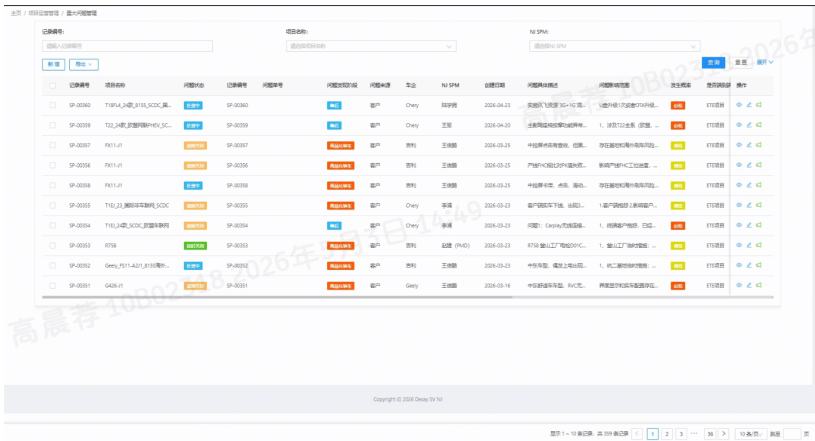


图 6-7 重大问题管理界面

同时集成了飞书卡片通知机制，在问题新增时自动推送消息至相关责任人，飞书卡片消息提醒效果如图6-8所示。这个模块还支持图表来可视化重大问题的分布情况，统计结果如图6-9所示，便于了解哪些项目的重大问题比较突出。

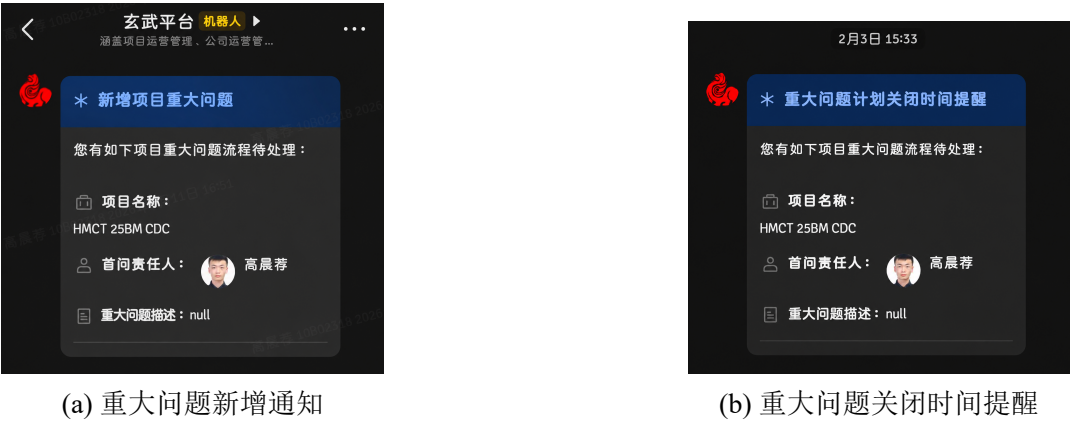


图 6-8 飞书消息通知



图 6-9 重大问题统计分析

表 6.5 重大问题管理接口测试用例

编号	测试用例	测试步骤	预期结果	测试结果
TC14	列表查询与筛选	进入列表页 → 按项目名称/状态/日期等条件筛选 → 搜索	列表按条件过滤，分页正常	符合预期
TC15	新增问题与飞书通知	填写表单 → 提交	保存成功，飞书卡片通知自动发送	符合预期
TC16	编辑问题	选择记录 → 修改字段 → 保存	更新成功，数据持久化	符合预期
TC17	查看详情	点击查看图标	抽屉弹窗展示全部字段信息（只读）	符合预期
TC18	导出 Excel	点击导出	下载xlsx文件	符合预期
TC19	通知 SPM	点击重大问题右侧小喇叭	飞书卡片发送至指定 SPM	符合预期

6.2.5 各功能的边界测试

表 6.6 边界测试用例表

编号	测试用例	测试步骤	预期结果	测试结果
TC20	查询 BU 工时分布	不输入组织 id 或者 不输入年月	饼图部分展示一个空 数据页面	符合预期
TC21	修改项目交付历史	点击编辑, 不修改内 容, 然后查看交付历 史	交付时间轴上节点没 有新增	符合预期
TC22	项目名称重复 新增	新增项目时使用已 存在的项目名	返回”项目名称重复, 请检查”	符合预期
TC23	飞书 receivers 为空	发送飞书消息传空 receivers 列表	返回飞书通知发送失 败	符合预期

6.3 非功能测试

在确保系统功能满足业务需求的基础上, 系统的稳定性、安全性以及性能响应同样是企业级应用不可忽视的重要指标。本节将从边界异常处理、数据越权访问防护以及海量数据并发查询等维度展开非功能性评估。

6.3.1 安全性测试

安全测试主要看重大问题部分系统是否支持权限划分, 即普通用户、项目经理以及管理员登录后, 对重大问题查询的可见性是否符合预期的效果。用例及测试结果如表6.5所示。

表 6.7 安全性测试用例表

编号	测试用例	测试步骤	预期结果	测试结果
TC24	重大问题可见性	由于没有管理员的 账号密码, 选择在代 码部分将登录用户 uid 分别设置为管理 员,SPM, 普通用户	管理员可以看到所有 重大问题, 其他人只能 看到和自己相关的重 大问题	符合预期

6.3.2 性能测试

系统在开发过程中主要在缺陷查询以及 BU 工时统计部分遇到了加载较慢的问题, 因此本小节针对这两个部分进行性能测试。对于深分页来说, 主要是测试系统响应时间随着页号的变化趋势; 工时则是看 SQL 的执行计划。

(1) 缺陷查询深分页性能测试

本文采用 AOP 切面方式对方法进行耗时测量。通过在待测方法上添加自定义注解，系统在方法执行前后分别记录时间戳，并计算两者差值得到单次查询耗时。该方式避免了在业务方法内部直接插入计时代码，从而降低测试逻辑对业务逻辑的侵入。例如对深分页的测试流程如下，为减少偶然波动对测试结果的影响，每组测试重复执行若干次，并取平均耗时作为最终结果。

Algorithm 1: 深分页查询性能对比测试流程

Input: 页码集合 P , 分页大小 s , 重复次数 n
Output: 平均耗时 $\bar{T}_{base}$ 与 $\bar{T}_{opt}$

```

1 foreach 页码  $p \in P$  do
2   设置分页参数: 页码为  $p$ , 分页大小为  $s$ ;
3   for  $i \leftarrow 1$  to  $n$  do
4     执行普通分页方法并记录耗时  $T_{base}^i(p)$ ;
5     执行优化分页方法并记录耗时  $T_{opt}^i(p)$ ;
6   end
7   计算传统分页平均耗时  $\bar{T}_{base}(p)$ ;
8   计算优化分页平均耗时  $\bar{T}_{opt}(p)$ ;
9 end
    
```

深分页优化前后性能对比如图6-10所示。测试结果表明，当页号逐渐增大时，传统的直接分页查询耗时上升较快。而采用优化后的分页方案，其查询响应时间稳定在百毫秒级别。

在进行优化前，随着页号的增加，响应时间增加多少，经过优化后，响应时间并没有大幅增加，即使页号偏移到5000，响应时间也在可接受范围内。

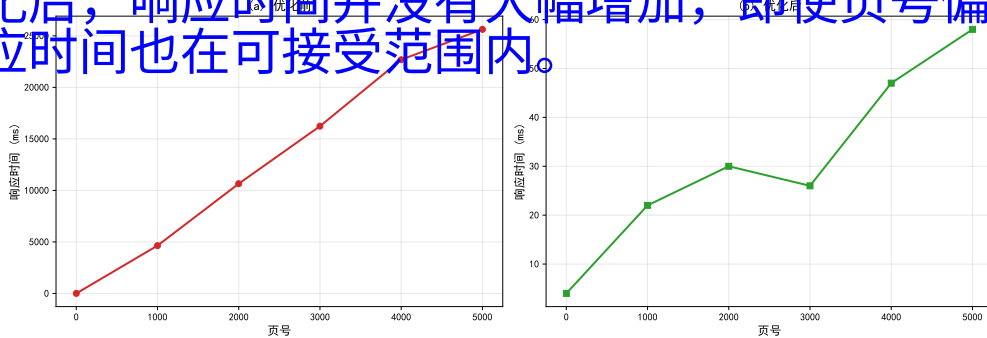


图 6-10 优化前后页面加载性能对比

改成一张图，

(2) BU 工时性能测试

在第五章的分析部分，提到了部门投入 BU 工时统计功能需要按照月份汇总各业务部门的工时投入情况。初始实现在筛选月份时使用 DATE_FORMAT 函数对工时日期字段进行格式化处理，再与目标年月进行比较。该写法虽然能够实现月份筛选，但由于在索引字段上使用了函数计算，数据库难以直接利用 work_date 字段上的索引进行范围定位，可能导致大规模数据下扫描行数增加。针对上述问题，本文将月份筛选条件改写为日期范围查询。优化前后的执行计划对比如表 6.8 所示。

表 6.8 查询执行计划对比

版本	type	key	Extra
优化前	ALL	NULL	Using where; Using temporary; Using filesort
优化后	range	idx_wh_work_date	Using index condition; Using temporary; Using filesort

由表可知，优化前查询的访问类型为 ALL，表示数据库需要对 tb_sync_wh 表进行全表扫描且 key 列为 NULL。其主要原因在于对 work_date 这个索引使用了函数操作，使得该索引失效了。

优化后，查询条件被改写为基于日期区间的范围查询，访问类型由 ALL 变为 range，Extra 中出现了 Using index condition，并实际使用了 idx_wh_work_date 索引。这说明优化后的查询能够利用日期索引缩小扫描范围，从而降低数据库在工时明细表上的数据访问成本。

6.4 本章小结

本章对实现的企业级汽车数字化运营平台进行了全面、系统的测试。首先明确了测试所依赖的软件基础环境；其次，通过测试用例覆盖了各个核心功能模块，验证了系统业务逻辑的准确性；最后，通过开展边界测试、安全性验证及性能测试等非功能性测试，证明了系统各项指标均达到了预期的设计要求，具备较高的可用性与安全性。

第七章 总结与展望

7.1 工作总结

玄武数字化运营平台是笔者实习所在公司的一个研发效能平台，主要面向汽车电子软件项目管理场景。本文围绕该平台的部分功能模块，完成了需求分析、系统设计、编码实现和功能测试等工作。

本文第一章分析了数字化运营平台的开发背景和国内外研发现状，介绍了论文的组织结构。第二章对系统用到的 SpringBoot、Vue、MySQL、Redis、ECharts 等关键技术做了简要说明。第三章从普通员工、项目经理和部门经理三类角色出发，对 BU 工时统计分析、项目基本信息管理、缺陷管理和重大问题管理四个模块进行了功能性需求分析，同时从性能、容错、安全和易用性等方面梳理了非功能性需求。第四章描述了系统的 BS 四层架构，将整个平台按业务维度划分为系统管理、公司运营管理和项目运营管理三大模块，并说明笔者负责的后两个模块中各功能点的职责。第五章对四个模块的详细设计做了阐述，包括类图、时序图、关键业务逻辑和数据库 E-R 模型。第六章展示了各模块的实现效果，通过功能测试用例和非功能性测试对系统进行了验证，其中针对缺陷深分页查询采用了延迟关联方案，优化后查询时间稳定在百毫秒级。

笔者在项目中具体负责的工作有：公司运营管理模块下的 BU 工时统计分析，支持按组织和年月维度查询各部门在业务单元上的工时投入，通过饼图和趋势图展示工时占比与变化趋势，利用 Redis 缓存减少数据库查询压力；项目运营管理模块下的项目基本信息维护，实现了项目的增删改查、条件筛选、Excel 导出和交付历史追溯，采用主表加历史表的双写方式保留修改记录；缺陷管理模块，对接了 Trinity、Jira 和 BugClose 三个外部平台，实现缺陷数据的统一查询和聚类统计分析；重大问题管理模块，实现了问题的新增、编辑、状态自动判定、飞书卡片通知和统计分析功能。

7.2 工作展望

本文设计实现的数字化运营平台已基本满足企业项目管理的日常需求，但在实际应用过程中仍暴露出一些不足之处，有待进一步优化和完善。未来的改进方向主要包括以下几个方面：

- (1) 当前系统基于 Spring Boot 2.1 和 Vue2 构建，框架版本比较老旧，部分核心依赖已停止维护更新，存在潜在的安全风险。后续可以将前后端框架升级至 Spring Boot 3.x，以便于引入 Springai 来让系统集成大模型服务，在缺陷分析、风险评估和项目报告生成等场景中探索智能化辅助能力。并迁移至 Java21 长期支持版本，因为这个版本的 JDK 提出了虚拟线程，非常适用于 Web 应用程序这种 I/O 密集型场景，前端每次启动速度都比较慢，可以考虑迁移至 Vue 3 配合 Vite 构建工具，以获得更好的性能和更完善的

生态支持。

- (2) 目前平台已具备公司运营管理和项目运营管理两大功能域，能够为企业提供基本的数字化支撑。但从企业整体运营管理的视角来看，尚未覆盖企业运营管理的全部层级，后续可以在这个平台上继续扩展部门运营管理子系统，对部门级的人力资源效能、项目承接能力和团队协同效率做更细化的分析；同时可以考虑建设研发运营管理子系统，把需求吞吐量、代码提交频率、构建成功率等研发过程指标纳入平台，帮助团队发现开发流程中的瓶颈；另外还可以朝战略运营管理方向拓展，将各业务域的数据做更高层次的汇总，为管理层提供经营决策方面的数据支持。

参考文献

- [1] IBM. What is a software-defined vehicle?[EB/OL]. 2025 [2026-04-01]. <https://www.ibm.com/cn-zh/think/topics/software-defined-vehicle>.
- [2] ISO 24089:2023 Road Vehicles – Software Update Engineering[A/OL]. International Organization for Standardization. 2023 [2026-03-30]. <https://www.iso.org/standard/77796.html>.
- [3] ISO/SAE 21434:2021 Road Vehicles – Cybersecurity Engineering[A/OL]. International Organization for Standardization and SAE International. 2021 [2026-05-10]. <https://www.iso.org/standard/70918.html>.
- [4] VDA Working Group 13. Automotive SPICE® Process Reference Model and Process Assessment Model, Version 4.0[A/OL]. VDA Quality Management Center. 2023 [2026-04-10]. <https://vda-qmc.de/wp-content/uploads/2023/12/Automotive-SPICE-PAM-v40.pdf>.
- [5] VERHOEF P C, BROEKHUIZEN T, BART Y, et al. Digital Transformation: A Multidisciplinary Reflection and Research Agenda[J/OL]. Journal of Business Research, 2021, 122: 889-901 [2026-05-14]. <https://doi.org/10.1016/j.jbusres.2019.09.022>.
- [6] Project Management Institute. A Guide to the Project Management Body of Knowledge (PMBOK® Guide) – Eighth Edition and The Standard for Project Management[M/OL]. Project Management Institute, 2025 [2026-05-17]. <https://www.pmi.org/standards/pmbok>.
- [7] REBUGLIO M, ZENEZINI G, OTTAVIANI F M. Project Management Information Systems: A Systematic Review[J/OL]. Procedia Computer Science, 2025, 256: 1739-1747.
- [8] Atlassian. Jira Documentation[EB/OL]. Atlassian. [2026-04-21]. <https://confluence.atlassian.com/jira>.
- [9] Asana. Build an App with Asana[EB/OL]. Asana. [2026-04-21]. <https://developers.asana.com/docs/overview>.
- [10] DORA Team. Accelerate State of DevOps Report 2024[R/OL]. Google Cloud. 2024 [2026-05-16]. <https://dora.dev/research/2024/dora-report/>.
- [11] Renee. 禅道文档[EB/OL]. 禅道软件（杭州）有限公司. (2025-09-04) [2026-04-21]. <https://www.zentao.net/book/zentaopms/503.html>.
- [12] TAPD. 开放平台文档[EB/OL]. TAPD. (2024-07-25) [2026-04-21]. <https://open.tapd.cn/document/api-doc/>.
- [13] ISO/IEC/IEEE 42010:2022 Software, Systems and Enterprise – Architecture Description[A/OL]. International Organization for Standardization. 2022 [2026-05-21]. <https://www.iso.org/standard/74393.html>.
- [14] LINDHOLM T, YELLIN F, BRACHA G, et al. The Java® Virtual Machine Specification: Java SE 8 Edition[A/OL]. Oracle America, Inc. (2015-02-13) [2026-04-27]. <https://docs.oracle.com/javase/specs/jvms/se8/html/index.html>.
- [15] Spring. Auto-configuration – Spring Boot Reference Documentation[EB/OL]. 2025 [2026-04-03]. <https://docs.spring.io/spring-boot/reference/using/auto-configuration.html>.

- [16] Spring. Build Systems: Starters – Spring Boot Reference Documentation[EB/OL]. 2025 [2026-05-10]. <https://docs.spring.io/spring-boot/reference/using/build-systems.html>.
- [17] Spring. Externalized Configuration – Spring Boot Reference Documentation[EB/OL]. 2025 [2026-04-18]. <https://docs.spring.io/spring-boot/reference/features/external-config.html>.
- [18] Spring. Packaging Your Application for Production – Spring Boot Reference Documentation[EB/OL]. 2025 [2026-05-25]. <https://docs.spring.io/spring-boot/reference/using/packaging-for-production.html>.
- [19] Vue.js Team. Introduction – Vue.js v2 Guide[EB/OL]. 2023 [2026-05-09]. <https://v2.vuejs.org/v2/guide/>.
- [20] Apache ECharts. Get Started with Apache ECharts[EB/OL]. 2025 [2026-04-01]. <https://echarts.apache.org/handbook/en/get-started/>.
- [21] Oracle. MySQL 8.4 Reference Manual: Introduction to InnoDB[EB/OL]. 2024 [2026-05-11]. <https://dev.mysql.com/doc/refman/8.4/en/innodb-introduction.html>.
- [22] Oracle. MySQL 8.4 Reference Manual: InnoDB Multi-Versioning[EB/OL]. 2024 [2026-04-09]. <https://dev.mysql.com/doc/refman/8.4/en/innodb-multi-versioning.html>.
- [23] Redis. Redis Data Types[EB/OL]. 2025 [2026-04-16]. <https://redis.io/docs/latest/develop/data-types/>.
- [24] Redis. Redis Persistence[EB/OL]. 2025 [2026-05-23]. https://redis.io/docs/latest/operate/oss_and_stack/management/persistence/.
- [25] Redis. Scripting with Lua[EB/OL]. 2025 [2026-05-13]. <https://redis.io/docs/latest/develop/programmability/eval-intro/>.
- [26] Baomidou. Quick Start – MyBatis-Plus[EB/OL]. (2025-09-30) [2026-05-12]. <https://baomidou.com/en/getting-started/>.
- [27] NICTER D. Efficient MySQL Performance[M/OL]. O'Reilly Media, 2021 [2026-04-04]. <https://www.oreilly.com/library/view/efficient-mysql-performance/9781098105082/>.
- [28] BOTROS S, TINLEY J. High Performance MySQL[M/OL]. 4th ed. O'Reilly Media, 2021 [2026-04-18]. <https://www.oreilly.com/library/view/high-performance-mysql/9781492080503/>.